POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Rolling-Horizon $A^\star$ Planning and Nonlinear MPC for Map-Free Navigation of a Legged Robot

Project Report
062047 – Numerical Optimization for Control

Professor:
Lorenzo Mario Fagiano

Group Members:
Lorenzo Ortolani
10751686

Francesco Pedrini
10787879

Academic year:
2025-2026

Abstract: Autonomous navigation in an unknown, cluttered environment can be posed as a pair of nested optimization programs solved at every control cycle: a discrete shortest-path problem on a LiDAR-derived occupancy grid, and a continuous nonlinear program (NLP) that turns the resulting geometric reference into dynamically feasible velocity commands. Here, we analyze the map-free navigation of a legged robot, primarily a Unitree G1 humanoid, with a Unitree Go2 quadruped as the second platform. The high level plant consists of non-linear forward kinematic dynamics, valid for each legged robot abstracted as a body-frame holonomic agent, with first-order actuator lag for modeling the low level dynamics latency, yielding a six-state, three-input discrete model; the nonlinear MPC tracker is formulated in CasADi and solved by IPOPT at 8 Hz. Because that abstraction leaves the two machines with the same states and the same inputs, the stack is substantially identical on both: what changes between them is the tuning, the assumed lag and the quantitative conclusions that follow, not the formulation. The eleven cost weights of that NLP are firstly tuned by heuristics, but can be further improved by an offline calibration using a per-axis Bayesian scheme: a one-dimensional Gaussian process per weight, six closed-loop mission rollouts each, swept in order of influence with every optimum carried forward, for 67 rollouts in total. That campaign is the one part of the study it was possible to run only on the quadruped, since there was no possibility to deploy it on the humanoid for time reasons. The reported results therefore refer to the Go2, even if a completely analogous approach could be applied to the humanoid as well. The remaining analyses are instead mainly focused on the G1. Most of the design decisions in the project derive from theoretical considerations and quantified outcomes, rather than being taken in an arbitrary way.

Key-words: Nonlinear Model Predictive Control, Interior-Point Optimization, Rolling-Horizon Planning, $A^\star$ Search, Map-Free Legged Robot Navigation, Bayesian Optimization

Contents

Acronyms

AD	algorithmic differentiation
AMO	Adaptive Motion Optimization
DDS	Data Distribution Service
EKF	extended Kalman filter
GP	Gaussian process
IK	inverse kinematics
IMU	inertial measurement unit
KF	(ordinary, i.e. non-extended) Kalman filter
KKT	Karush–Kuhn–Tucker
L-BFGS	limited-memory Broyden–Fletcher–Goldfarb–Shanno
LICQ	linear independence constraint qualification
LiDAR	light detection and ranging
LTI	linear time-invariant
MPC	model predictive control
MUMPS	multifrontal massively parallel sparse direct solver
NLP	nonlinear program
PD	proportional–derivative
PPO	proximal policy optimization
QP	quadratic program
RK2	second-order Runge–Kutta (mid-point)
RL	reinforcement learning
ROS	Robot Operating System
SLAM	simultaneous localization and mapping
SQP	sequential quadratic programming

Notation

All physical quantities carry SI units. Vectors are columns; $\|\cdot\|$ without subscript is the Euclidean norm. Subscript k indexes a node inside the prediction horizon, subscript j an obstacle. The symbol $\lambda_{\min}(\cdot)$ denotes the smallest eigenvalue of its argument, and is not related to the multipliers λ.

State, input and model

x	state	$\tau_{v_x}, \tau_{v_y}, \tau_\omega$	actuator time constants [s]
u	commanded body twist	$\nu_{v_x}, \nu_{v_y}, \nu_\omega$	discrete lag coefficients
p_x, p_y, ψ	world-frame pose [m], [rad]	Δt	sampling time [s]
v_x, v_y, ω	body twist [m/s], [rad/s]	N	prediction horizon [steps]
t	continuous time [s]	N_c	control horizon [steps]

Environment and reference

$\mathcal{P}_k$	LiDAR point cloud at cycle k	δ	occupancy grid resolution [m]
$P(c)$	occupancy of cell c	σ	Gaussian inflation width [m]
x_g	goal position [m]	P_{thr}	occupancy hard-obstacle threshold
v_{ref}	cruise speed [m/s]		

Optimization program

$z = (X, U)$	decision variables	λ, μ	KKT multipliers (eq., ineq.)
J	objective	μ_b	interior-point barrier parameter
Q, R, R_{jerk}	tracking, effort, rate weights	ρ_T	terminal weight multiplier
d_{safe}	imposed clearance [m]	s_{jk}	constraint slack [m]
ρ_s	slack penalty weight	θ	cost-weight vector
ε_d	distance regulariser	ε_M	machine precision
$W_{\text{obs}}, \alpha_{\text{obs}}, r_{\text{obs}}$	obstacle penalty height, steepness, radius		

Locomotion layer

$q, \dot{q}$	joint positions, velocities	β	gait duty factor
$\mathcal{T}$	joint torques	ζ_i	stride phase of leg i
K_P, K_D	joint PD gains	$T_{\text{str}}, T_{\text{st}}, T_{\text{sw}}$	stride, stance, swing durations [s]
p_i^{f}	foot position of leg i [m]	L_i	step length of leg i [m]

1. Introduction

A mobile robot that must reach a goal in an environment it has never seen has no global map to plan on: at every instant it knows only what its sensors reported in the last few metres. The classical answer is to decouple the problem into a geometric planner and a low-level tracker, but if the two layers ignore each other the planner produces paths the actuators cannot execute, and the tracker produces commands the geometry does not allow. Model Predictive Control (MPC) resolves the conflict by construction: it optimizes a finite sequence of future inputs subject to the actual dynamics and the actual constraints, and re-optimizes at every cycle [18]. The price is computational for the motion planner point of view: a nonlinear program must be solved to sufficient accuracy inside the sampling period, every period, on the robot. However, this rolling horizon path planning, allows the robot to avoid simultaneously localizing and mapping the environment. The presence of only a localization module through an EKF and the lack of complete SLAM, enables a less computationally expensive navigation stack, which is an important contribution of this report.
The goal is to analyze legged robotics navigation stack, such as a Unitree Go2 dog robot or a Unitree G1 humanoid robot. The stack is deployed and tested entirely on the G1 robot and previously developed on a Unitree Go2 quadruped, whose locomotion layer, the CHAMP gait controller [9] on the simulated robot and the vendor's sport-mode controller on the physical one, exposes a body-frame velocity interface. The same code has since been ported to a Unitree G1 humanoid walking under a reinforcement-learning locomotion policy [12], which is what makes the "robot-agnostic" claim testable, due to the simple high level holonomic kinematic model derived from both systems. Section 2.3 describes those two locomotion layers and the abstraction that makes them interchangeable. Two programs are solved at every planning cycle and are the object of the analysis:

- a *discrete* shortest-path program, an $A^\star$ search over a Gaussian occupancy grid rebuilt from the current LiDAR scan, whose solution is a geometric reference and whose rolling-horizon target is the intersection of the ray to the global goal with the grid boundary;
- a *continuous* nonlinear program, a six-state, three-input MPC with actuator-lag dynamics, a smooth obstacle barrier and box-constrained inputs, formulated once symbolically in CasADi [2] and solved by the interior-point method IPOPT [22] at 8 Hz.

On top of these two runtime programs sits a third, solved once offline: the *black-box* program of choosing the eleven cost weights of the MPC. Weight tuning is the practical bottleneck of any deployed MPC, since depending on the environment, MPC performance can vary significantly. The problem is that the closed-loop performance of the MPC is a gray-box function of unknown parameters, namely the weights for each cost term. The automatic weights tuning problem is implemented with Gaussian-process surrogates [17, 21] for finding the optimal weight vector $\hat{\theta}$ of the MPC cost function.

2. Problem description and modeling

2.1. System overview

The stack is a chain of ROS 2 nodes that closes the loop between a 3-D LiDAR and the body-frame velocity interface of the locomotion controller:

1. an **odometry bridge** that converts the extended-Kalman-filter (EKF) pose estimate into a single pose topic and differentiates it, with an exponential filter, into a body-frame velocity estimate (the robot does not publish a usable body-frame twist);
2. a **mapping module** that rebuilds a local Gaussian occupancy grid from the newest scan at 1 Hz and maintains a sparse world-frame cell map of confirmed obstacle cells, on which the geodesic target rule of Section 3.2 plans; its purpose is precisely to repair the failure mode measured in Section 5.4.1;
3. the $A^\star$ **local planner**, which solves the discrete program of Section 3.2 on that grid;
4. the **nonlinear MPC tracker**, which solves the NLP of Section 3.4 at 8 Hz and publishes a setpoint;

Two design decisions shape everything that follows. First, the locomotion layer is treated as an ideal velocity-tracking device with a time constant: we never model legs, contacts or gait, and the robot is abstracted as a planar holonomic agent, Sections 2.2 and 2.3 describe what that layer actually does on each of the two platforms, and why the same abstraction survives the difference between them. Second, the map is *local and volatile*. The occupancy grid covers a fixed 12×12 m window centred on the robot and is rebuilt from scratch at every planning cycle, so the planning problem is genuinely map-free, and the geometric reference the MPC tracks is only valid for one grid width ahead. Section 5.4.1 measures what this choice costs, and Section 5.4.2 what it takes to repair it.

2.2. The controlled platforms

The same stack drives two robots whose mechanics could hardly be more different: a Unitree Go2 quadruped (Fig. 1(a)) and a Unitree G1 humanoid (Fig. 1(b)). The Go2 has twelve actuated joints, three per leg; the G1 has twenty-nine, of which the locomotion layer of Section 2.3.2 drives fifteen, the twelve leg joints and the three waist joints, while the arms are held at a fixed posture. Neither joint set is ever addressed by the high level trajectory tracking MPC controller. Both platforms are commanded through the same channel, a body-frame twist $u = (v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, \omega^{\mathrm{cmd}})$, and both carry a 3-D LiDAR whose returns are the only exteroceptive input of the stack.

(a) Unitree Go2 quadruped

(b) Unitree G1 humanoid

Figure 1: The two platforms on which the same navigation stack is deployed. (a) The Go2 quadruped, with its LiDAR mounted on the head. (b) The G1 humanoid, to which the stack was ported without changing the optimization program: only the constants of the actuator-lag model (12) and the command mapping of Section 2.3.2 differ.

Table 1: The two platforms as the controller sees them. The rows above the first rule are properties of the machine, those between the rules are properties of the locomotion layer that receives the MPC command, and those below are the ones that enter the optimization program: the prediction model (12), the admissible input set and the horizon. The three time constants take the same value on both machines and were identified on neither: what the two platforms share is the structure of the abstraction and not a measurement, and that is the concrete form the "robot-agnostic" claim takes.

	Unitree Go2	Unitree G1
Morphology	quadruped	humanoid
Actuated joints	12 (3 per leg)	29 (15 driven by the gait policy)
Exteroception	Unitree L1 LiDAR	Livox MID-360 LiDAR
Locomotion layer	CHAMP gait controller (vendor sport mode on hardware)	AMO reinforcement-learning policy
Layer update rate	200 Hz	50 Hz
Command interface	body-frame twist u	body-frame twist u, remapped (Section 2.3.2)
Abstraction used by the MPC	first-order velocity lag, $\tau = 0.00100$ s in the deployed profile (Section 2.5)	
Admissible input set	$v_x \in [-0.20, 0.40]$ m/s, $\lvert v_y \rvert \le 0.20$ m/s, $\lvert \omega \rvert \le 0.40$ rad/s	
Horizon	$N = 15$, $\Delta t = 0.35$ s (5.25 s of preview), re-solved at 8 Hz	

2.3. Low-level locomotion layers

The controller analysed in this report never commands a joint. Its decision variable is a body-frame twist, and between that twist and the ground sits a platform-specific layer that turns it into joint commands at 200 Hz on

the Go2 and 50 Hz on the G1. Only two properties of that layer reach the optimization problem: the twist is tracked with a first-order lag rather than instantaneously, and the tracking is fast enough, relative to the 8 Hz MPC cycle, for the two layers to be designed separately. This section describes the two locomotion controllers to the depth needed to justify those two properties and no further. Throughout, $q \in \mathbb{R}^{n_j}$ denotes the vector of joint positions ($n_j = 12$ on the Go2, $n_j = 29$ on the G1), $\mathcal{T} \in \mathbb{R}^{n_j}$ the vector of joint torques, and $K_P, K_D \succ 0$ the diagonal proportional and derivative gain matrices of the joint-level loop; the symbol τ is reserved for the actuator time constants of Section 2.5.

2.3.1 Model-based gait generation: CHAMP on the Go2

CHAMP [9] is an open-source implementation of the hierarchical trot controller developed for the MIT Cheetah [8, 11]. It is a purely feedforward pipeline: a pattern modulator imposes the footfall sequence, a trajectory planner turns each leg's phase into a foot position, inverse kinematics (IK) turns that position into joint angles, and a joint-level impedance loop tracks them. CHAMP is the locomotion layer of the simulated robot.

Pattern modulation. Let t denote time. Each leg $i \in \{\mathrm{LF}, \mathrm{RF}, \mathrm{LH}, \mathrm{RH}\}$ carries a normalised stride clock

$$\zeta_i(t) = \left(\frac{t}{T_{\mathrm{str}}} - \zeta_i^0 \right) \bmod 1 \in [0, 1), \qquad T_{\mathrm{str}} = T_{\mathrm{st}} + T_{\mathrm{sw}}, \tag{1}$$

where T_{st} and T_{sw} are the stance and swing durations and ζ_i^0 is the phase offset of leg i. Leg i is in stance while $\zeta_i < \beta$ and in swing otherwise, with duty factor $\beta = T_{\mathrm{st}}/T_{\mathrm{str}}$. The deployed configuration uses $T_{\mathrm{st}} = T_{\mathrm{sw}} = 0.25$ s, hence $T_{\mathrm{str}} = 0.5$ s and $\beta = 0.5$, and $\zeta^0 = (0, \frac{1}{2}, \frac{1}{2}, 0)$: the diagonal pairs move together, which is the trot.

Where the command enters. The commanded twist does not act on the legs directly; it acts on the *step geometry*, through the Raibert foot-placement heuristic [16]: a foot is placed half a step ahead of its nominal position, so that sweeping backwards during the stance interval T_{st} displaces the trunk by exactly the commanded amount. Let $\bar{p^{\mathrm{f}}}_i \in \mathbb{R}^3$ be the nominal (zero-command) position of foot i in the body frame. Translating the nominal stance by the half-step $\frac{1}{2} T_{\mathrm{st}} \, (v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, 0)^\top$ and rotating it about the vertical axis by the half-stance yaw χ_g gives the foot displacement

$$\Delta p_i^{\mathrm{f}} \;=\; R_z(\chi_g) \Big(\bar{p^{\mathrm{f}}}_i + \tfrac{1}{2} T_{\mathrm{st}} \left(v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, 0 \right)^\top \Big) - \bar{p^{\mathrm{f}}}_i, \qquad \chi_g = 2 \sin\!\Big(\tfrac{1}{4} T_{\mathrm{st}} \, \omega^{\mathrm{cmd}} \Big) \approx \tfrac{1}{2} T_{\mathrm{st}} \, \omega^{\mathrm{cmd}}, \tag{2}$$

with $R_z(\cdot)$ the elementary rotation about the vertical axis.[1] The step length and the heading of the foot trajectory of leg i follow as

$$L_i = 2 \big\| \Delta p_{i}^{\mathrm{f}} \big\|_2, \qquad \chi_i = \operatorname{atan2}\left(\Delta p_{i,y}^{\mathrm{f}}, \, \Delta p_{i,x}^{\mathrm{f}} \right), \tag{3}$$

the factor 2 turning the half-step into the full stride of the foot. Equations (2) and (3) are the whole of the command path: the three numbers the MPC publishes are compressed into one length and one angle per leg, and nothing else of the command reaches the robot.

Foot trajectories and impedance. With phase and step length fixed, each foot follows a planar curve in its own trajectory frame, rotated by χ_i and added to the nominal stance: $p_i^{\mathrm{f}} = \bar{p^{\mathrm{f}}}_i + (\, \xi \cos\chi_i, \, \xi \sin\chi_i, \, \eta \,)^\top$, where ξ and η are the longitudinal and vertical coordinates of the curve. In stance, parametrised by the normalised stance phase $\zeta_i^{\mathrm{st}} = \zeta_i / \beta \in [0, 1)$, the foot sweeps backwards along a shallow arc that presses into the ground by h_{st},

$$\xi = \tfrac{L_i}{2} \left(1 - 2\zeta_i^{\mathrm{st}} \right), \qquad \eta = -h_{\mathrm{st}} \cos\!\left(\frac{\pi \, \xi}{L_i} \right); \tag{4}$$

in swing, parametrised by $\zeta_i^{\mathrm{sw}} = (\zeta_i - \beta)/(1 - \beta) \in [0, 1)$, it follows a Bézier curve of degree 11 through twelve control points $b_j \in \mathbb{R}^2$,

$$(\xi, \eta) = \sum_{j=0}^{11} \binom{11}{j} \big(\zeta_i^{\mathrm{sw}} \big)^j \big(1 - \zeta_i^{\mathrm{sw}} \big)^{11-j} \, b_j, \tag{5}$$

whose horizontal spread is scaled by L_i and whose vertical spread is scaled by the swing height h_{sw}. Figure 2 collects these quantities on a single leg. The deployed gait uses $h_{\mathrm{sw}} = 0.04$ m, $h_{\mathrm{st}} = 0.01$ m and a nominal

[1] The implementation obtains χ_g from the tangential half-step $\frac{1}{2} T_{\mathrm{st}} \, \omega^{\mathrm{cmd}} d_c$, with d_c the horizontal distance from the trunk centre to the nominal foot; d_c cancels, leaving (2). The small-angle form on the right is the yaw swept in half a stance interval, as expected.

trunk height $h_0 = 0.225$ m. A closed-form IK then maps each foot position to the three joint angles of its leg, $q_i^{\mathrm{ref}} = \mathrm{IK}_i(p_i^{\mathrm{f}})$, and a proportional–derivative (PD) law closes the loop at the joints,

$$\mathcal{T}_i = K_P\big(q_i^{\mathrm{ref}} - q_i\big) + K_D\big(\dot{q}_i^{\mathrm{ref}} - \dot{q}_i\big) \;=\; \boldsymbol{J}_i(q_i)^{\top} F_i, \tag{6}$$

where $\boldsymbol{J}_i$ is the foot Jacobian of leg i, set in bold to distinguish it from the MPC cost J and from the tuning objective $\mathcal{J}$ and F_i the equivalent force at the foot. The PD loop is in joint space and realises a virtual spring–damper at the foot, of Cartesian stiffness $\boldsymbol{J}_i^{-\top} K_P \boldsymbol{J}_i^{-1}$, so the leg absorbs terrain irregularity and impact by programmable compliance rather than by contact sensing or force control. This is what makes an open-loop gait generator survive on a real robot.

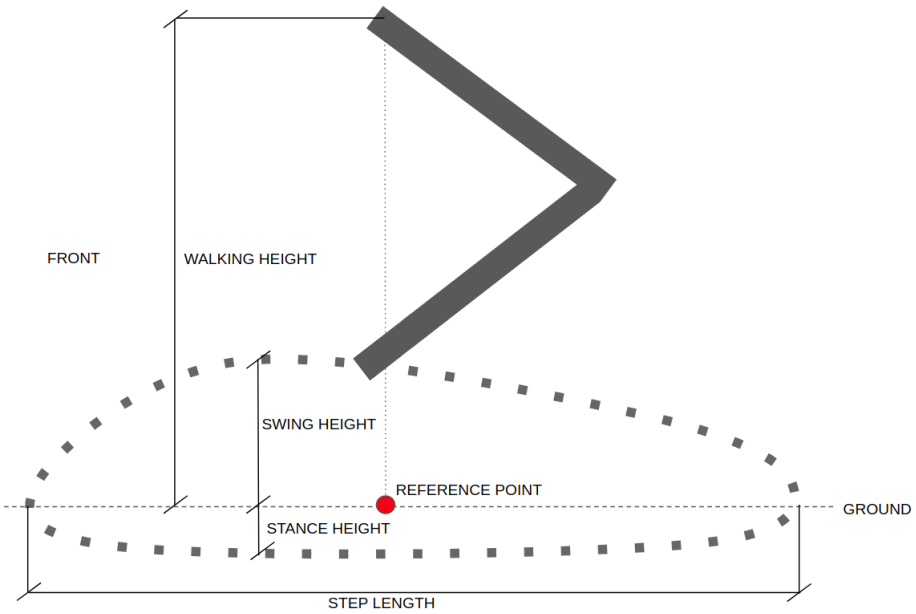


Figure 2: Geometry of the CHAMP gait on a single leg, seen from the side. The dotted closed curve is the cycle described by the foot in its trajectory frame: the upper branch is the swing Bézier (5), which clears the ground by the swing height h_{sw}, and the lower branch is the stance arc (4), which presses into the ground by the stance height h_{st} and sweeps backwards to carry the trunk forward. The horizontal extent of the cycle is the step length L_i of (3), the only channel through which the commanded twist reaches the leg; the reference point on the ground line is the nominal stance position $\bar{p}^{\mathrm{f}}_i$ about which the Raibert half-step (2) displaces the cycle, and the walking height is the nominal trunk height h_0. Adapted from the CHAMP documentation [9].

Low level joint controllers modelled as a lag. Three consequences propagate upstream, and they are the reason the model of Section 2.5 has the form it has. First, the twist enters only through the step geometry (3): no measured trunk velocity is fed back, so the layer is a feedforward map and the trunk velocity approaches the commanded one over roughly one stride rather than immediately. Second, that step geometry is re-evaluated only once per stride, so a command that changes faster than $1/T_{\mathrm{str}} = 2$ Hz is averaged, not tracked; a first-order lag with $\tau_{v_x} = 0.12$ s reaches 98% of its final value in $4\tau_{v_x} \approx T_{\mathrm{str}}$. This is the sense in which a single time constant of order 0.1 s describes the layer, and Section 2.5 is where that constant is separated from the one the prediction model actually carries. Third, the gait applies its own saturation, $|v_x^{\mathrm{cmd}}| \le 0.3$ m/s, $|v_y^{\mathrm{cmd}}| \le 0.25$ m/s and $|\omega^{\mathrm{cmd}}| \le 0.5$ rad/s in the deployed configuration.

2.3.2 Learned whole-body control: the AMO policy on the G1

On the humanoid the same interface is produced by a neural network instead of a gait generator. The deployed controller is AMO (Adaptive Motion Optimization) [12], a reinforcement-learning (RL) policy trained in simulation for the G1 and its twenty-nine degrees of freedom (DoF). AMO is deliberately *not* a walking controller: it is a loco-manipulation policy, trained to track torso height, torso orientation and upper-body targets while walking, on a hybrid dataset in which an offline trajectory-optimization module supplies whole-body references that are kinematically consistent with the commanded end-effector motion. The navigation stack exercises only its locomotion subset, the arms are held at the default posture and only the base command is driven, but the

policy is the one trained for the larger task, which is what makes the same platform usable for manipulation later without changing the layer the MPC talks to.

Structure. At each tick k of the 50 Hz policy loop, the network maps an observation to an action,

$$a_k = \pi(o_k) \in \mathbb{R}^{15}, \qquad o_k = \big(\omega_k^{\mathrm{b}},\ \hat{g}_k,\ \tilde{u}_k,\ q_k - q^{\mathrm{def}},\ \dot{q}_k,\ a_{k-1}\big), \tag{7}$$

where $\omega_k^{\mathrm{b}} \in \mathbb{R}^3$ is the body angular velocity, $\hat{g}_k \in \mathbb{R}^3$ the gravity direction expressed in the body frame (the tilt an inertial measurement unit (IMU) provides without a global heading), $q_k, \dot{q}_k$ the measured joint positions and velocities of the 23 observed joints, q^{def} the default posture, a_{k-1} the previous action, and $\tilde{u}_k$ the command. The action is not a torque: it is an offset on the default posture, which the same joint-level PD law as (6) converts into torque,

$$q_k^{\mathrm{ref}} = q^{\mathrm{def}} + s_a\, a_k, \qquad \mathcal{T}_k = K_P\big(q_k^{\mathrm{ref}} - q_k\big) + K_D\big(0 - \dot{q}_k\big), \tag{8}$$

with s_a a fixed action scale and a zero reference joint velocity. The learned part of the controller is therefore exactly the map from observation to PD setpoint; the stabilising loop underneath it is the same fixed impedance as on the quadruped. The policy π is obtained by proximal policy optimization (PPO) [20], maximising the expected discounted return $\mathbb{E}\big[\sum_k \gamma^k \mathcal{R}_k\big]$ with $\gamma \in (0,1)$ and a stage reward $\mathcal{R}_k$ that pays for tracking the commanded twist and penalises posture deviation, joint effort and action rate; masses, friction, gains and latencies are randomised during training so that the fixed policy transfers to hardware [19].

2.3.3 The cascade and its abstraction seam

Fig. 3 places the two locomotion layers in the control structure they belong to. The stack is a cascade of five loops whose rates are separated by roughly a decade each: the $A^\star$ reference is recomputed at 1 Hz, the MPC at 8 Hz, the proportional law that turns the MPC look-ahead setpoint into a body twist at 20 Hz, the locomotion layer at 50–200 Hz, and the joint PD loop at the motor-driver rate. That separation is the standard justification for designing each layer against a static abstraction of the one below it, and the abstraction used here is deliberately minimal: three decoupled first-order lags, Eq. (12), whose time constants are chosen in Section 2.5. Both locomotion layers, the hand-designed one and the learned one, are approximated by the same small set of numbers.

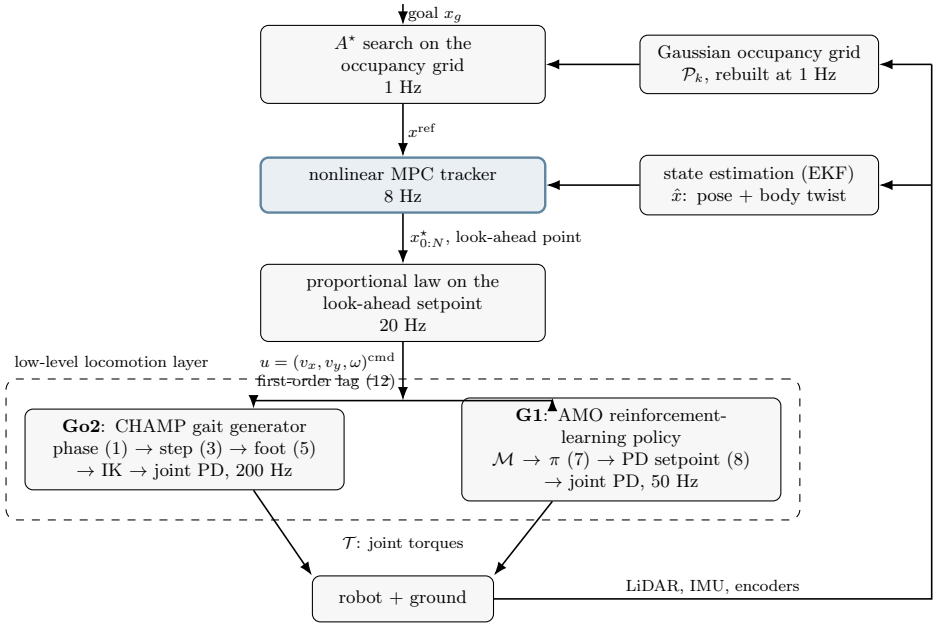


Figure 3: Cascaded structure of the controller, from the goal down to joint torques. The rates fall by roughly a decade at every seam (1 Hz, 8 Hz, 20 Hz, 50–200 Hz, motor-driver rate), which is what allows the layers to be designed separately, and the two branches inside the dashed box, a hand-designed gait generator and a learned policy, are interchangeable precisely because they present the same twist interface u.

2.4. Environment representation

Let $\mathcal{P}_k = \{p_i \in \mathbb{R}^3\}_{i=1}^{N_k}$ be the LiDAR point cloud available at cycle k, projected on the plane. The occupancy grid is a square of side $2h_w$ ($h_w = 6$ m) centred on the robot position, discretised with resolution $\delta = 0.20$ m into $n_c = 2h_w/\delta = 60$ cells per axis. The occupancy probability of cell c is a Gaussian tail of the distance to the nearest return,

$$P(c) \;=\; 1 - \Phi\!\left(\frac{d_{\min}(c, \mathcal{P}_k)}{\sigma}\right), \qquad d_{\min}(c, \mathcal{P}_k) = \min_{p \in \mathcal{P}_k} \|c - p\|_2, \tag{9}$$

with Φ the standard normal cumulative distribution function,

$$\Phi(x) \;=\; \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-t^2/2}\, \mathrm{d}t \;=\; \frac{1}{2}\left(1 + \operatorname{erf}\!\left(\frac{x}{\sqrt{2}}\right)\right) \in (0, 1), \tag{10}$$

and $\sigma = 0.31$ m. Equation (9) is an *inflation* model: it is a smooth, monotone map from clearance to cost, whose one parameter σ sets the width of the obstacle halo. Cells with $P(c) \geq P_{\text{thr}} = 0.10$ are hard obstacles. The pair is chosen together: by the blocking radius $d_{\text{block}} = \sigma\, \Phi^{-1}(1 - P_{\text{thr}})$ of Section 3.2 it implies a hard keep-out of 0.40 m, which is the 0.35 m body radius of the G1 with a margin. The left panel of Fig. 4 shows the trade-off governed by σ: a small σ leaves doorways open but hugs walls, a large one closes gaps the robot could physically cross.

Because the grid is rebuilt from scratch, its construction cost is the dominant cost of the planning layer, $O(n_c^2\, |\mathcal{P}_k|)$; vectorised over 60×60 cells and $\sim$180 returns it stays an order of magnitude below the geodesic field of Section 3.2, which is the one expensive part of the planning layer, so at the 1 Hz replanning rate the whole discrete layer is negligible against the 8 Hz MPC. Rebuilding is also what rules out incremental repair schemes such as D* Lite [10], which amortise replanning by reusing the previous search tree: here the graph is discarded and re-centred at every cycle, so there is nothing to reuse.

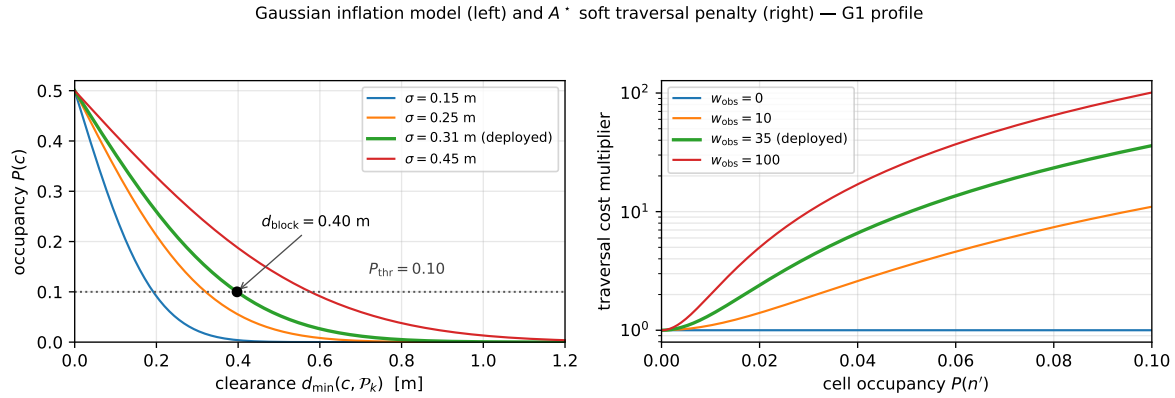


Figure 4: Left: the Gaussian inflation model (9) for four values of σ, the deployed one in bold, with the hard threshold $P_{\text{thr}} = 0.10$ and the blocking radius $d_{\text{block}} = \sigma\, \Phi^{-1}(1 - P_{\text{thr}}) = 0.40$ m it implies. Right, on a logarithmic scale: the soft traversal-cost multiplier $1 + w_{\text{obs}}(P/P_{\text{thr}})^2$ of the $A^\star$ edge cost (15), which is what actually keeps the path away from walls; the deployed value is $w_{\text{obs}} = 35$. Both panels are drawn from the deployed profile `planner_params_g1.yaml` rather than from hand-set values.

2.5. Plant model and state-space formulation

The MPC model must predict where the robot will be. Commanding a legged platform a body-frame velocity does not produce that velocity instantaneously: the locomotion controller responds with a measured time constant of $\tau \approx 0.10$–0.15 s, which over a 5 s horizon accumulates into a prediction error of several tens of centimetres if neglected. The model therefore augments the usual planar kinematic state with the three velocity states, giving

$$x \;=\; \begin{bmatrix} p_x & p_y & \psi & v_x & v_y & \omega \end{bmatrix}^\top \in \mathbb{R}^6, \qquad u \;=\; \begin{bmatrix} v_x^{\text{cmd}} & v_y^{\text{cmd}} & \omega^{\text{cmd}} \end{bmatrix}^\top \in \mathbb{R}^3, \tag{11}$$

where (p_x, p_y) and ψ are the world-frame pose and (v_x, v_y, ω) the body-frame twist actually realised by the platform. In continuous time the actuator behaves as three decoupled first-order lags driven by the commands,

$$\dot{v}_x = \frac{u_1 - v_x}{\tau_{v_x}}, \qquad \dot{v}_y = \frac{u_2 - v_y}{\tau_{v_y}}, \qquad \dot{\omega} = \frac{u_3 - \omega}{\tau_{\omega}}, \tag{12}$$

while the pose obeys the holonomic body-to-world rotation

$$\dot{p}_x = v_x \cos\psi - v_y \sin\psi, \qquad \dot{p}_y = v_x \sin\psi + v_y \cos\psi, \qquad \dot{\psi} = \omega. \tag{13}$$

The response of the locomotion layer is a property of the machine, and Section 2.3.1 places it at $\tau \approx 0.1$ s: the step geometry is re-evaluated once per stride, so the trunk velocity approaches the commanded one over roughly T_{str} and a command faster than $1/T_{\text{str}} = 2$ Hz is averaged rather than tracked. That is a quadruped-gait argument; on the humanoid, whose policy exposes no equivalent clock, the same order of magnitude is here assumed rather than derived.

Whether to carry that constant inside the prediction model is a separate decision, and it buys two things. The first is fidelity: the prediction knows that a commanded velocity takes time to establish itself. The second is regularisation, and it is the one that binds. A model that believes velocity builds gradually has little to gain from swinging the input between one node and the next; a model that believes the platform is instantaneous has every incentive to, and the resulting command stream approaches the rate of the layer that has to execute it, at which point the two loops stop being separated in frequency.

For the sake of simplicity, and to keep things simple when simulating the robots in ROS, the deployed profile carries neither of the previous two aspects, setting $\tau_{v_x} = \tau_{v_y} = \tau_\omega = 0.00100$ s: at $\Delta t = 0.35$ s the discrete coefficient of Section 3.3 is 1.000000 and (19) collapses to $v_{k+1} = u_k$, so the model contains no actuator dynamics and the three velocity states are one-step delays. The two omissions do not cost the same. Fidelity is cheap here: at the physically motivated $\tau = 0.12$ s the coefficient would be 0.9459 rather than 1, and replayed over horizons from the recorded mission the two models disagree by a median of 2.4 mm at the end of a 5.25 s horizon and 8.4 mm at worst, which is larger than the mid-point truncation error of Section 3.3, but amounts to only 1.2% of one grid cell. Regularisation is relocated: by Section 2.6 the optimizer is a reference generator and $u_0^\star$ is never sent to the locomotion layer, which receives a twist produced at 20 Hz by a proportional law from a filtered look-ahead setpoint, while the input-rate penalty R_{jerk} of (23) acts inside the cost. The frequency separation the lag was there to protect is therefore enforced downstream of the NLP and in the objective, where both are explicit and tunable, rather than implicitly through a modelling constant that is also carrying prediction fidelity.

The drawbacks are that the prediction believes the platform reaches any commanded velocity within one node and the terminal-feasibility argument of Section 3.4.6 holds for a reason that belongs to the profile rather than to the robot. Identifying τ under physics would settle whether it should be restored. The three symbols are kept distinct in the notation even though the profile gives them one value, because such an identification would separate them: forward travel is the axis the gait is designed around, while the lateral and yaw channels are produced by the same low-level layer from the same twist command. That identification is a different class of problem, optimal system identification, and is outside the scope of this report.

Finally, model (12)–(13) is holonomic: unlike a differential-drive vehicle, the platform can command a lateral velocity, so no nonholonomic constraint appears and the input set is a box.

2.6. Navigation problem statement

Let $x_g \in \mathbb{R}^2$ be the goal. The navigation task is the receding-horizon optimal control problem

$$\begin{aligned}
\min_{u_0,\dots,u_{N-1}} \quad & J(x_{0:N}, u_{0:N-1}, \mathcal{P}_k, \theta) && && \text{(14a)}\\
\text{s.t.} \quad & x_{t+1} = f(x_t, u_t), && t = 0,\dots,N-1, && \text{(14b)}\\
& x_0 = \hat{x}(t_k), && && \text{(14c)}\\
& u_t \in \mathcal{U}, && t = 0,\dots,N-1, && \text{(14d)}\\
& d(x_t, \mathcal{P}_k) \geq r_{\min}, && t = 0,\dots,N, && \text{(14e)}
\end{aligned}$$

solved at every control instant t_k. Two properties of the way (14) meets the robot are worth stating before the design is read, because both are deviations from the textbook receding-horizon implementation.

The sampling time of the model is not the period of the loop. The step Δt in (14b) is the *prediction* step, 0.35 s in the deployed profile, while the program itself is re-solved at 8 Hz. The horizon is therefore re-planned about 2.8 times per predicted node, and the first predicted node already lies 0.35 s of open loop ahead of the state that produced it. The two quantities are independent design variables and are treated as such throughout.

Only u_0 would be applied, but it is not. The tracker does not publish $u_0^\star$. It walks the predicted trajectory $x_{0:N}^\star$ out to a fixed look-ahead distance and publishes the point it reaches as a geometric setpoint; a proportional law downstream converts that setpoint into the body twist actually sent to the locomotion layer, at 20 Hz. The optimizer is thus used as a *reference generator* rather than as a controller, and the loop closed on the robot is the cascade of that law with the locomotion policy rather than (14) itself. Section 5.1 bypasses the stage

deliberately, so that the numerical results measure the optimization layer and not the tuning of the law beneath it.
Three features of (14) then determine the whole design:

- **The goal is not reachable within the horizon.** With $N\Delta t = 5.25$ s and a cruise speed of 0.27 m/s the horizon spans about a metre and a half, while the goal may be tens of metres away and outside the sensed region altogether. The cost (14a) therefore cannot be a distance to x_g; it must be a distance to a *reference trajectory* produced by a separate, cheaper program that looks further ahead. That program is the $A^\star$ search of Section 3.2.
- **The obstacle set is a point cloud, not a polytope.** Constraint (14e) is non-convex, and its description changes at every cycle. It is relaxed into a smooth penalty (Section 3.4.2), which keeps the NLP smooth and always feasible at the price of losing the hard safety guarantee.
- **The cost is parametrised by θ.** The eleven weights in θ are not physical quantities; they are design variables of a second-level optimization problem (Section 4.5).

3. Formulation of the optimization program

3.1. Division of labour: motion planning against trajectory optimisation

The two programs formulated in this chapter answer different questions, and confusing them is the most common misreading of the stack. *$A^\star$ (Section 3.2) is where obstacle avoidance is decided.* It searches the actual occupancy grid, so it is the only layer that ever asks which side of an obstacle to pass on, and it answers with a discrete, globally consistent path: a graph search needs no initial guess and cannot get trapped in the wrong homotopy class the way a gradient step can. *The NMPC (Section 3.4.1) never re-asks that question.* Its job is narrower and local: given the $A^\star$ path as a reference, compute the sequence of commands that tracks it as closely as the dynamics (30a) and the input envelope (30c)–(30d) allow, while keeping the commanded twist smooth and dynamically feasible. The clearance term J_{obs} of (24) does add an obstacle cost to that program, but it is a local safety margin around a path already known to be clear, not a second avoidance layer: it is a smooth, soft penalty precisely because the hard, discrete decision has already been made upstream (Section 3.4.4 measures how close it comes to a hard constraint, and at what cost). Splitting the problem this way is also what keeps the NMPC solvable at 8 Hz, since writing the discrete choice into the continuous program would need integer variables, Section 3.2 makes that argument in full.
The split is not free of failure modes of its own: $A^\star$ can commit to a bad choice that the NMPC then follows faithfully (Section 5.4.1), and the two layers can each leave their own local minimum in the smooth program (Section 5.2). But the roles stay fixed throughout, motion planning happens once per replanning cycle, in the discrete layer; trajectory optimisation happens every control cycle, in the continuous one, and every design choice in the rest of this chapter follows from which of the two questions it is answering.

3.2. Reference generation: $A^\star$ as a discrete program

Section 2.6 established that the goal lies far outside the horizon, so the cost of (14a) must measure the distance to a reference trajectory rather than to x_g. That reference is produced once per replanning cycle by a discrete search on the occupancy grid of Section 2.4. Cells above the occupancy threshold are removed from the graph; the remaining 8-connected edges carry the cost

$$g(n \to n') \;=\; c_{\text{move}} \cdot \delta \cdot \left[1 + w_{\text{obs}} \left(\frac{P(n')}{P_{\text{thr}}}\right)^2\right], \qquad c_{\text{move}} \in \{1, \sqrt{2}\}, \tag{15}$$

and a standard $A^\star$ search returns the cheapest path between two cells [7]. The normalised quadratic in (15) is what produces medial-axis-like paths: a cell just below the hard threshold costs $(1 + w_{\text{obs}})$ times an open cell, so the search prefers the middle of a corridor to a shorter path that grazes a wall, while a doorway is never closed outright. The Euclidean heuristic is admissible because the bracket is bounded below by 1, so no edge can be cheaper than its Euclidean length; the search is therefore an exact $A^\star$ and returns a cost-optimal path on the current occupancy grid.

What the discrete layer is for. Choosing *which side* of an obstacle to pass on is a discrete decision. Deforming a trajectory that passes on the left into one that passes on the right sweeps it through the obstacle: the feasible set is disconnected, one component per side, and a gradient-based solver stays in the component its initial guess falls into. Writing the choice into the NLP would require binary variables and a mixed-integer

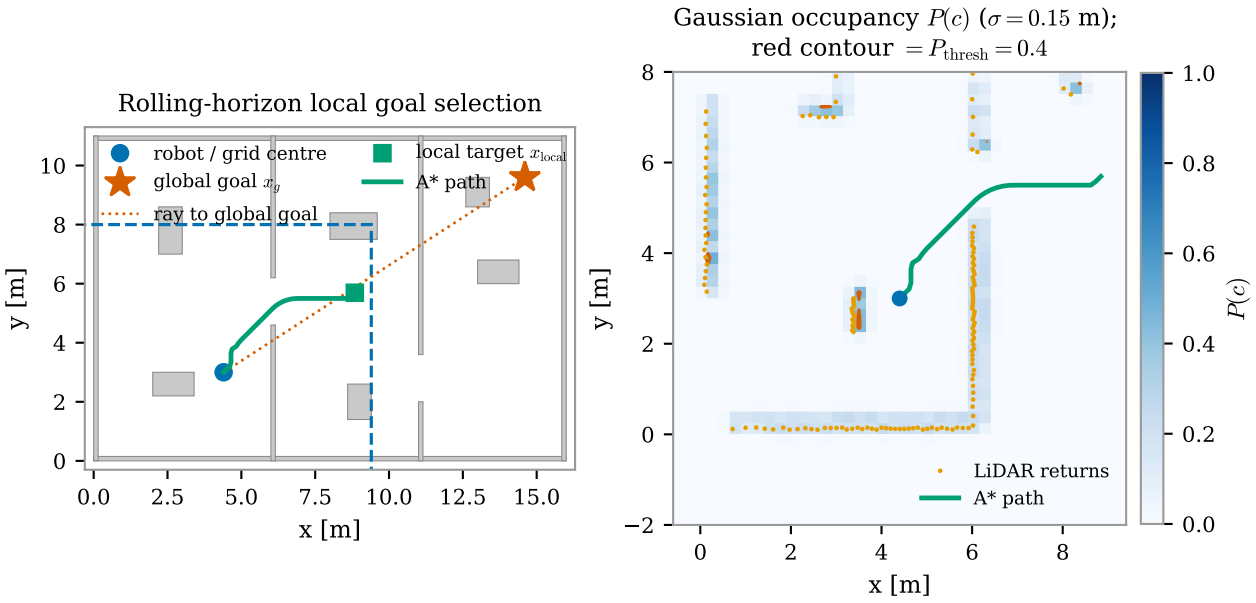


Figure 5: Rolling-horizon local goal selection (left) and the corresponding occupancy grid with the $A^\star$ path (right). The window $\mathcal{W}(\hat{x})$ is re-centred on the robot every replanning cycle rather than covering the whole map, so when the global goal x_g falls outside it a local target is projected onto the window boundary along the ray towards x_g and $A^\star$ plans only up to that point. This is what keeps the discrete search cheap regardless of how far the goal actually is, at the cost of the target rule discussed next.

program, whose solution time is not compatible with a loop at 8 Hz, whereas a graph search represents it natively and costs a few milliseconds on a 60×60 grid. Delegating it to $A^\star$ is what leaves the continuous program smooth and box-constrained, the structure that Sections 4.1 and 4.2 exploit. The price is that the choice precedes the MPC and cannot be revised by it: when the discrete layer *does not commit*, the smooth program inherits two competing basins of its own (Section 5.2); when it *commits on poor information*, the MPC follows it faithfully into a dead end (Section 5.4.1).

A convex obstacle needs nothing further: the ray to the goal leaves the obstacle on one side or the other, (15) routes around it, and the next cycle finds the robot closer than the last. Instead concave obstacles break that guarantee, this is the reason why the next three components exist.

Memory: the accumulated map. The LiDAR reports only what is in view, so a wall leaving the field of view is forgotten by the raw scan and the robot replans as if it had never seen it. Returns are therefore accumulated in world frame as a sparse set of cell indices at the resolution of the planning grid, each stamped with the time it was last confirmed; entries expire after 20 s, so a moving obstacle is forgotten once it has moved. Stored cells inside the window are merged with the current scan before the grid of Section 2.4 is built, so (15) plans against walls the sensor can no longer see.

This is a necessary addition, but not sufficient. In the failure of Section 5.4.1 the robot has already observed the closed end of the concavity and still drives back into it, which is the useful diagnosis: the information is in the map, but the rule that selects the target throws it away.

Choosing the target inside the window. The search of (15) runs on the local grid and the global goal is normally outside it, so a target on the window has to be selected first. This is a selection over the free cells of the window boundary,

$$x_{\text{local}} \;=\; \operatorname*{arg\,min}_{c \,\in\, \partial\mathcal{W}(\hat{x}) \cap \mathcal{F}_k} \Big[\, d_k\big(c,\, x_g\big) \;+\; w_\tau\, \tau_k(c) \,\Big], \tag{16}$$

where $\mathcal{F}_k$ are the free boundary cells, d_k is a distance to the goal and τ_k penalises cells already visited without progress. Only boundary cells are candidates, because the boundary is where the direction of travel is decided; an interior cell would stop the robot half a window short of the edge for no reason. The lever is the *target* and not the edge cost: raising w_{obs} in (15) makes a concavity expensive to traverse, not unattractive to aim at.

The rule the stack was initially designed on does not solve (16). It simply takes the last free cell along the ray

from the robot to the goal,

$$x_{\text{local}} = \begin{cases} \text{cell}(x_g), & x_g \in \mathcal{W}(\hat{x}), \\ \text{ray}(\hat{x} \to x_g) \cap \partial\,\mathcal{W}(\hat{x}), & \text{otherwise}, \end{cases} \tag{17}$$

which is cheap, needs no map, and is memoryless and purely directional: in front of a concavity the line to the goal enters it, so the target does too. Replacing the projection by the arg min of the straight-line distance does not repair this. At the mouth of a horseshoe (Figure 6) the projection selects a cell 30.2 m of travel from the goal and the Euclidean arg min a different cell 30.4 m away, equally wrong, because in a straight line both are close to the goal; where two candidates tie, the choice is settled by numerical noise and reverses at the next cycle.

The metric: geodesic distance. The deployed rule keeps (16) and replaces d_k with the *geodesic* distance to the goal over the accumulated map. Write ρ for the cell side, $\mathcal{B}_k$ for the cells lying within $d_{\text{block}} = \sigma\,\Phi^{-1}(1 - P_{\text{thr}})$ of an accumulated obstacle point, $\mathcal{F}_k$ for the rest of the grid, and c_g for the cell containing x_g. The field is the unique solution of the shortest-path fixed point

$$d_k(c, x_g) = \begin{cases} 0, & c = c_g, \\ \min\limits_{c' \in \mathcal{N}(c) \cap \mathcal{F}_k} \Big\{ d_k(c', x_g) + \rho\,\omega(c, c') \Big\}, & c \in \mathcal{F}_k \setminus \{c_g\}, \\ +\infty, & c \in \mathcal{B}_k, \end{cases} \tag{18}$$

with $\mathcal{N}(c)$ the eight-cell neighbourhood and $\omega = 1$ on the four axial edges, $\omega = \sqrt{2}$ on the four diagonal ones. The weights are non-negative, so (18) is solved exactly by one Dijkstra wavefront propagated *from* c_g in $\mathcal{O}(|\mathcal{F}_k| \log |\mathcal{F}_k|)$, and cells the wavefront never reaches keep $d_k = +\infty$.
Space not yet observed counts as free, which is why $\mathcal{B}_k$ is built from observed points alone. This is the optimistic assumption of frontier exploration, and it is what makes the field correct itself as the sensor discovers: while the far end of a corridor is unseen the wavefront runs through it and entering is the right decision; as soon as that end is observed the same cell becomes distant and the target is dropped. The robot does not avoid concavities on principle, it tries each one once. A candidate with $d_k = +\infty$ is discarded rather than scored with a Euclidean fallback, which would readmit the metric at fault; if no candidate is reachable the planner falls back to (17), which at least moves the robot towards unknown ground. In Figure 6 the geodesic arg min selects a cell 7.4 m from the goal, on the far side of the obstacle's arm, and $A^\star$ routes out of the concavity instead of further into it. The memory term $w_\tau \tau_k$ of (16) is implemented but carries zero weight in the deployed profile, for the reason measured in Section 5.4.2.

Commitment between replanning cycles. Two boundary cells on opposite sides of an obstacle can carry almost the same geodesic cost while belonging to different homotopy classes. The arg min then alternates between them from one cycle to the next and the robot sways on the spot without taking a decision about which side to explore. The deployed rule therefore retains the candidate that best continues the previous choice unless a competitor beats it by 2.0 m of geodesic cost, like adding a sort of istheresis effect when the planner want to switch decision from one side to the other. The rule also clears that memory whenever the global goal changes, since a target inherited from a previous mission is simply the wrong direction. It is a margin on the cost and not a lock: a genuinely better route still wins. Where the smooth program of Section 5.2 resolves the same tie by breaking symmetry numerically, the discrete layer resolves it by declining to switch without evidence.
The geodesic field is the one expensive part of the planning layer, because the wavefront covers the whole known map rather than the window alone: it measures a median of 80 to 121 ms per replanning cycle depending on the world, with a worst case of 440 ms, a tenth to a half of the 1 Hz replanning period, and no load at all on the 8 Hz MPC. The resulting grid path is finally resampled and smoothed with a moving average before it is handed to the MPC, to remove the zig-zag that the 8-connected discretisation introduces.

3.3. Model discretization

With sampling time Δt, the lag subsystem (12) is linear and time-invariant, so it is discretised *exactly* under a zero-order-hold input. Defining $\nu_i = 1 - e^{-\Delta t/\tau_i}$ for $i \in \{v_x, v_y, \omega\}$, the velocity update is the exact solution of (12) over one step,

$$v_{x,k+1} = (1 - \nu_{v_x}) v_{x,k} + \nu_{v_x} u_{1,k}, \qquad \omega_{k+1} = (1 - \nu_\omega)\omega_k + \nu_\omega u_{3,k}, \tag{19}$$

and analogously for v_y with ν_{v_y}. No approximation is involved here and none should be claimed: this is the Lagrange formula for a linear time-invariant (LTI) system under a piecewise-constant input, and it is exact at

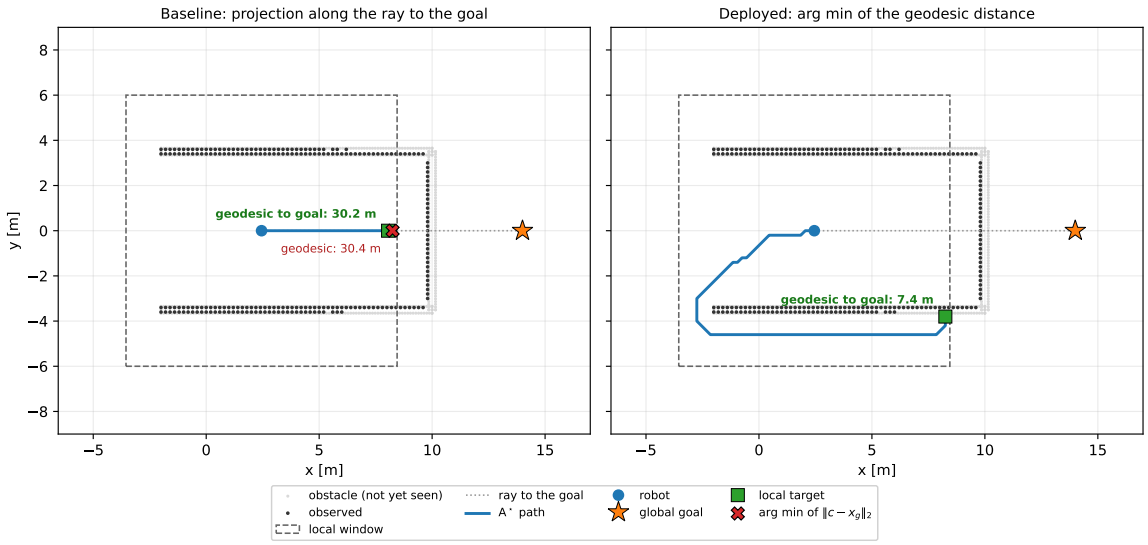


Figure 6: The same scene, the same window and the same observed map; only the metric of (16) changes. The robot sits at the mouth of a horseshoe and the map is the one it has actually accumulated by driving in, not the full world (light grey), which is what makes the failure reproducible. Left: the deployed projection (17) puts the target at the closed end, and the arg min of the Euclidean distance over the same free boundary cells lands beside it; the two rules differ, the outcome does not, because both are $\sim$30 m of travel from the goal. Right: with the geodesic d_k the target moves to the far side of the lower arm, 7.4 m away, and the $A^\star$ solution leaves the concavity. Drawn from the deployed planner and profile, not sketched.

the sampling instants. On the deployed G1 profile it is straightforward, since $\nu = 1$ reduces (19) to $v_{k+1} = u_k$; the statement is retained because it is the part of the model that would carry the weight on hardware, where τ is real.

The pose, by contrast, is genuinely nonlinear and must be integrated numerically. Two possibilities are implemented and compared:

$$\text{explicit Euler:} \quad p_{k+1} = p_k + R(\psi_k)\, v_{k+1}\, \Delta t, \tag{20}$$

$$\text{mid-point (RK2):} \quad p_{k+1} = p_k + R\big(\psi_k + \tfrac{1}{2}\omega_{k+1}\Delta t\big)\, v_{k+1}\, \Delta t. \tag{21}$$

Both evaluate the rotation on the *post-lag* velocity v_{k+1}, a semi-implicit choice that costs nothing and removes the one-step delay that an explicit scheme would introduce between a command and the predicted displacement. The mid-point rule differs from Euler by one addition inside the argument of the rotation.

The results shown in the following compare two integrators outside the optimizer, which is only meaningful if the map assembled inside the NLP is the same one. That is checked rather than assumed: a single transition is extracted from the CasADi program built by the tracker, by reconstructing $f(x_0, u_0)$ from the first six equality constraints, and compared with the reference implementation of (20) and (21). The residual is zero to machine precision for both schemes.

Error against the discretization scheme. The global error of each scheme was first fitted against the step size on three motion regimes, over a 2.80 s window, with the exact solution available in closed form: at constant body velocity the pose traces a circular arc. The measured orders are 1.00 and 2.00, exactly the first and second order the truncation analysis predicts, and the result is identical across regimes.

It is worth noting that a constant-velocity arc is not the regime in which the MPC works. The optimizer applies a *different* input at every node, and over a real horizon the commanded yaw rate changes sign.

The comparison is repeated on 12 horizons taken from a bag of data recorded while G1 is navigating in a mapless MuJoCo world and re-solved with the deployed profile, each integrated with its own optimal input sequence. The step is varied by subdividing the prediction interval while the input stays constant over each Δt, which keeps an order measurable on a real trajectory; the reference is the exact arc on the same post-lag velocity sequence, so the only difference between the three trajectories is how $R(\psi)$ is evaluated.

The orders survive the change of regime (1.00 and 2.00), but the error does not. At the deployed step the median is 2.08×10^{-2} m for Euler against 3.66×10^{-4} m for the mid-point rule, a factor 57 where the synthetic arc reports 114; the tails are 3.69×10^{-2} m and 7.16×10^{-4} m. The gap is the regime and not an inconsistency:

Table 2: Median position error at the end of the horizon, over 12 horizons re-solved from a bag of data recorded during the navigation, against the exact arc on the same velocity sequence. The first row is the deployed step; the others subdivide the prediction interval without changing the input.

Δt [s]	error, Euler [m]	error, mid-point [m]	ratio
0.3500	2.08×10^{-2}	3.66×10^{-4}	57
0.1750	1.04×10^{-2}	9.14×10^{-5}	113
0.0875	5.18×10^{-3}	2.28×10^{-5}	227
0.0437	2.59×10^{-3}	5.71×10^{-6}	453
0.0219	1.29×10^{-3}	1.43×10^{-6}	906

over these horizons the commanded yaw rate spans 0.54 rad/s, so the per-step errors partially cancel each other out, while the arc accumulates them along a single turn. Measuring on the synthetic regime alone would have overestimated the benefit by a factor of two.

The modelling floor. One number bounds the discussion. Holding the velocity at its post-lag value across the interval (the semi-implicit choice made in (20) and (21)) neglects a displacement of 8.01×10^{-4} m per horizon, because the model's velocity rises towards the command instead of arriving at it. That is larger than the mid-point truncation error itself. Below that level refining the pose scheme doesn't improve the results: a different modelling choice already dominates and the second order is not the binding approximation.

Closed-loop analysis. Prediction fidelity and executed cost are different quantities. Running the two schemes on the same worlds with everything else fixed moves the median closed-loop cost by at most 2.3%, and the sign changes from world to world, the signature of run-to-run variation rather than of an improvement.

Table 3: Closed-loop cost with the two integrators, same profile otherwise. A positive Δ means the mid-point rule is cheaper.

world	median cost, Euler	median cost, mid-point	Δ [%]
u_trap	163.5	167.2	-2.3
narrow_gap	110.7	108.2	2.3

The reason is structural and was stated in Section 2.6: only the first input is ever applied, and $A^\star$ replans at 1 Hz, so the accuracy of the far end of the prediction reaches the executed trajectory only indirectly. The mid-point rule is deployed because it is a strictly better prediction for the price of one addition, not because it was measured to steer better.

3.4. The nonlinear MPC program

3.4.1 Decision variables, cost, and the parametrisation of the program

The NLP is written in the *simultaneous* form, also called direct multiple shooting: both the state and the input sequences are decision variables and the dynamics enter as equality constraints,

$$z \;=\; \big(X, U\big), \qquad X = [x_0, \dots, x_N] \in \mathbb{R}^{6\times(N+1)}, \quad U = [u_0, \dots, u_{N-1}] \in \mathbb{R}^{3\times N}. \tag{22}$$

With $e_k = x_k - x_{\mathrm{ref},k}$ the cost is

$$J(z;\theta) \;=\; \sum_{k=0}^{N-1} \Big[e_k^\top Q\, e_k \;+\; u_k^\top R\, u_k \;+\; J_{\mathrm{obs}}(x_k) \Big] \;+\; \sum_{k=1}^{N-1} R_{\mathrm{jerk}} \|u_k - u_{k-1}\|_2^2 \;+\; e_N^\top Q_T\, e_N + J_{\mathrm{obs}}(x_N), \tag{23}$$

with $Q = \mathrm{diag}(Q_x, Q_y, Q_\psi, 0, 0, 0)$, $Q_T = \rho_T\, Q$ and $R = \mathrm{diag}(R_{v_x}, R_{v_y}, R_\omega)$. Three properties of (23) deserve comment.

The stage weight is only semi-definite. No velocity state is penalised: Q has rank 3 out of 6. The velocities are shaped indirectly, through the lag dynamics and through the input penalty R. This is what keeps the reference generator simple, since an $A^\star$ path carries no velocity information, and the price is that the cost contains no direct damping of the velocity states.

The rate penalty couples consecutive inputs. The term $R_{\text{jerk}}\|u_k - u_{k-1}\|_2^2$ is the discrete analogue of an input-rate penalty, and it is the only term that puts off-diagonal blocks between adjacent input stages in the Hessian. Its purpose is physical: a legged platform tracks a smooth velocity profile far better than a chattering one. Note that the sum starts at $k = 1$: the transition from the command applied on the previous cycle to u_0 is not penalised, so the first predicted move is free to jump. That is a deliberate omission (the program is re-solved at 8 Hz and would otherwise inherit a memory of its own past output), but it means the smoothness the term buys is smoothness *within* a horizon, not across consecutive ones.

The terminal cost is a scaled copy of the stage cost. $Q_T = \rho_T Q$ is a heuristic, neither the solution of a Riccati equation nor an exact cost-to-go, and like Q it acts only on the three position and heading components. ρ_T is therefore a tuning parameter and is treated as one, appearing among the eleven weights of θ in Section 4.5. (The gap between this heuristic and the true infinite-horizon cost-to-go of the linearised dynamics is not quantified in this report; doing so would require the Riccati comparison, which the semi-definite Q would first oblige us to regularise.) The stability argument is not carried by Q_T at all; what would carry it, and why the deployed profile does without it, is the subject of Section 3.4.6.

Comparison between multiple and single shooting. The standard argument for multiple shooting is that it keeps the Jacobian sparse and banded, where a condensed single-shooting formulation, in which states are eliminated by recursive substitution and so the inputs are the only variables, would produce a small but dense problem. A direct comparison between the two approaches can be performed, with the aim to show why a multiple-shooting approach was chosen.

Table 4: Multiple (M) against single (S) shooting on the same instance, at the deployed profile. Dimensions, densities and iteration counts are exact; each timing is the fastest of three cold solves, with no warm start. The table shows the comparison for the number of variables, number of constraints, the Hessian density, the number of iterations and the solve time needed by each method for different lengths of the prediction horizon N.

N	vars M / S	cons M / S	hess dens. [%] M / S	iter M / S	solve [ms] M / S
5	51 / 15	56 / 20	12.11/100.00	19 / 18	71 / 64
10	96 / 30	106 / 40	6.51/100.00	29 / 21	137 / 135
15	141 / 45	156 / 60	4.45/100.00	35 / 24	188 / 218
25	231 / 75	256 / 100	2.73/100.00	39 / 31	299 / 524
40	366 / 120	406 / 160	1.72/100.00	39 / 39	393 / 961
60	546 / 180	606 / 240	1.16/100.00	39 / 29	593 / 2106

The structural half of the claim holds exactly, and the Hessian is where it shows: at $N = 15$ the program shrinks from 141 variables and 156 constraints to 45 and 60, while the Hessian density goes from 4.45% to 100.00%, a full matrix. Condensing does trade a large banded problem for a small dense one.

If the last two columns together are analyzed, the reason why employing a multiple-shooting approach is beneficial becomes clear. At $N = 60$ the condensed form converges in *fewer* iterations than the sparse one, 29 against 39, and still takes three and a half times as long. The cost is therefore not in the number of Newton steps but in what each one costs, which is a factorisation of the Hessian, that as shown in the middle column goes from a few per cent dense to a full dense matrix.

It is worth noting that the result is not really valid in general: at $N = 15$ and $N = 40$ the two parametrisations converge to *different* minima, 6.7% and 5.8% apart in cost. That is not an implementation error (the program is non-convex and two parametrisations follow different optimization paths, so they can land in different basins), but it does mean that on those two rows, one of which is the deployed horizon, the timings compare solves that did not solve the same problem.

3.4.2 Obstacle avoidance as a smooth penalty

The clearance constraint (14e) is replaced, in the deployed configuration, by a penalty evaluated on the $M = 8$ nearest LiDAR returns $\{p_j\}$ inside a search radius of 2.0 m:

$$J_{\text{obs}}(x_k) \;=\; W_{\text{obs}} \sum_{j=1}^{M} \left[\underbrace{\tfrac{1}{2}\Big(1 - \tanh\big(\tfrac{\alpha_{\text{obs}}}{2}(d_{k,j} - r_{\text{obs}})\big)\Big)}_{\text{sigmoid zone}} + \underbrace{2\max\big(0,\; r_{\text{obs}} - d_{k,j}\big)^2}_{\text{penetration penalty}} \right], \tag{24}$$

with $d_{k,j} = \sqrt{(p_{x,k} - p_{j,x})^2 + (p_{y,k} - p_{j,y})^2 + \varepsilon_d}$ and $\varepsilon_d = 10^{-6}$ making the distance differentiable at zero, where the Euclidean norm is not. The sigmoid is written with tanh rather than as $1/(1 + e^{\alpha_{\text{obs}}(d - r_{\text{obs}})})$: the two are algebraically identical, but the logistic form overflows for large arguments while tanh saturates cleanly, and the argument does get large, since the sentinel padding described below puts absent returns at 10^3 m, which at $\alpha_{\text{obs}} = 6.00$ makes the exponent $\approx 6 \times 10^3$.

The hybrid structure is deliberate, and Fig. 7 is the argument for it. The sigmoid saturates at W_{obs} as $d \to 0$: on its own it is a *bounded* penalty, so a sufficiently strong tracking term can always buy its way through an obstacle. Worse for the solver, its gradient decays on both sides of r_{obs}, so the restoring force fades exactly where the robot most needs pushing out. At the deployed $\alpha_{\text{obs}} = 6.00$ the decay is mild (at two centimetres of clearance the sigmoid still retains about a third of the restoring force it has at $d = r_{\text{obs}}$), but it is governed by α_{obs} and it sharpens fast: at four times that steepness the same point retains essentially none. Steepening the barrier to improve avoidance is therefore precisely what destroys its gradient where avoidance has already failed. The quadratic term removes both defects inside the safety radius: it is unbounded, and its gradient grows linearly with penetration regardless of α_{obs}.

The price is the third panel. The curvature of (24) scales as $\alpha_{\text{obs}}^2 W_{\text{obs}}$, so a steep, heavy penalty injects large and rapidly varying eigenvalues into the Hessian precisely where the solution lives. The deployed tuning sits at the crossover: the peak curvature contributed by the sigmoid, $\approx 0.096\,\alpha_{\text{obs}}^2 W_{\text{obs}}$, is just below the $4W_{\text{obs}}$ contributed by the quadratic term, so neither dominates, and doubling α_{obs} already puts the sigmoid a factor of three above it, with the conditioning of the NLP following.

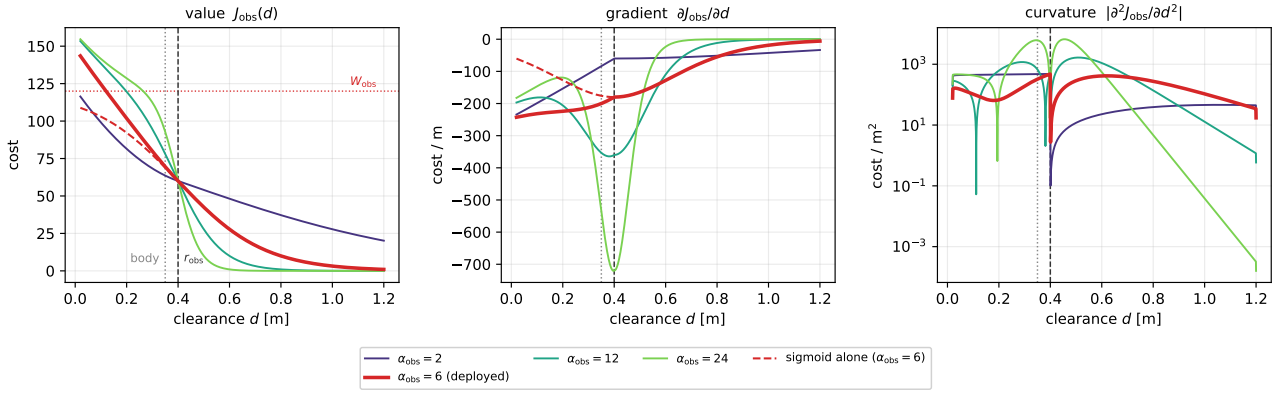


Figure 7: The hybrid penalty (24) against clearance, for four steepness values with the deployed $\alpha_{\text{obs}} = 6.00$ in bold ($W_{\text{obs}} = 120$, $r_{\text{obs}} = 0.40$ m; the dotted line marks the body radius). Dashed: the sigmoid zone alone at the deployed steepness, which saturates at W_{obs} and whose gradient fades inside the obstacle, the two defects the quadratic term repairs. Right, on a logarithmic scale: the curvature, which grows as α_{obs}^2 and is what degrades the conditioning of the NLP; the notches are sign changes. Evaluated through the same function the tracker builds into the NLP, not redrawn from the formula.

Relaxing a hard constraint into a penalty has three consequences worth stating explicitly. *(i)* The NLP is always feasible: no state constraint can render the problem unsolvable when the robot is already too close to an obstacle, which for a real-time loop with no fallback trajectory is a genuine robustness property. *(ii)* Safety is no longer guaranteed; (24) is not an exact penalty, so a finite W_{obs} admits a finite violation, and Section 3.4.4 makes that statement quantitative. A control-barrier formulation [1] would recover the guarantee, at the price of feasibility and of a harder NLP. *(iii)* The penalty is evaluated only at the $N + 1$ discrete nodes, so nothing forbids the continuous inter-sample trajectory from clipping a thin obstacle.

Point *(iii)* is worth checking rather than leaving as a caveat, because it is a one-line computation. Between two consecutive nodes the robot advances at most $v_{x,\max}\Delta t$, against a keep-out radius r_{obs}, so the penalty is meaningful only while

$$\frac{v_{x,\max}\,\Delta t}{r_{\text{obs}}} \;<\; 1. \tag{25}$$

On the deployed profile the numerator is 0.40×0.35, i.e. about 14 cm against 0.40 m of radius, so the ratio is 0.35: safe, but a third larger than before the input envelope was widened. It is reported rather than assumed because it is not invariant: it couples two parameters tuned for unrelated reasons (the sampling time of Section 5.3 and the input envelope of Table 1), and lengthening Δt to buy horizon walks it towards one. Above one the obstacle term is absent between nodes, and geometric avoidance falls entirely to the $A^\star$ layer on the inflated grid.

Sentinel padding. M is fixed. When fewer than M returns lie inside the search radius, the missing rows of the obstacle parameter are filled with a sentinel at 10^3 m, whose contribution to (24) is numerically zero. This apparently cosmetic detail is what allows the NLP to be built *once*: the number of terms, and therefore the sparsity pattern of the Jacobian and Hessian, never changes between cycles, so the symbolic factorisation and the warm start remain valid regardless of how cluttered the scene is.

3.4.3 Predicting moving obstacles: a constant-velocity Kalman filter

Every p_j in (24) is held fixed across the horizon: the node at $k = 0$ and the node at $k = N$ penalise distance to the *same* LiDAR return. For the industrial scenes of Section 5.1 that is not a simplification, it is exact, a shelf does not move between two nodes of a 5.25 s horizon. It stops being exact the moment an obstacle does: a person crossing at a brisk 1.2 m/s covers about $5.25 \times 1.2 \approx 6$ m over the horizon, so anchoring the penalty at the position measured when the cycle started places it a body length or more behind where the person actually is by the time the tail of the horizon is evaluated (Fig. 8).
Closing that gap needs an estimate of $\dot{p}_j$, not just p_j, and the same design already deployed for this exact problem on this robot's perception stack, a constant-velocity Kalman filter per tracked cluster, supplies it. Writing the track state as $o_j = (p_{j,x}, p_{j,y}, v_{j,x}, v_{j,y})^\top$, the predict step propagates it with the constant-velocity transition and a white-acceleration process noise Σ_a,

$$F = \begin{pmatrix} I_2 & \Delta t\, I_2 \\ 0 & I_2 \end{pmatrix}, \qquad \hat{o}_j^- = F\hat{o}_j, \qquad \Sigma_j^- = F\Sigma_j F^\top + \Sigma_a, \tag{26}$$

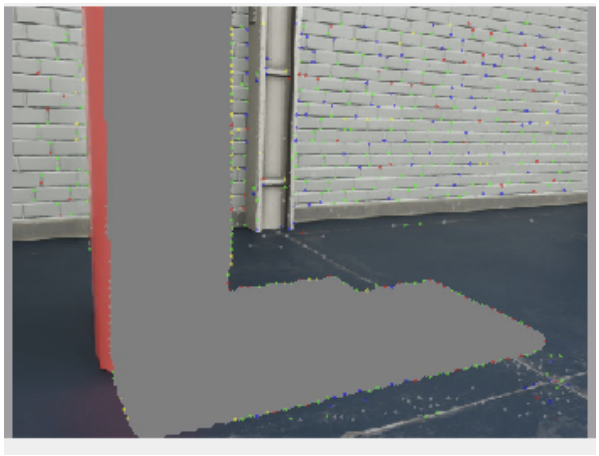
and the update corrects it against the next associated cluster centroid z_j, with $H = (I_2 \;\; 0)$ and measurement noise Σ_z,

$$S = H\Sigma_j^- H^\top + \Sigma_z, \quad K = \Sigma_j^- H^\top S^{-1}, \quad \hat{o}_j = \hat{o}_j^- + K(z_j - H\hat{o}_j^-), \quad \Sigma_j = (I - KH)\Sigma_j^-. \tag{27}$$

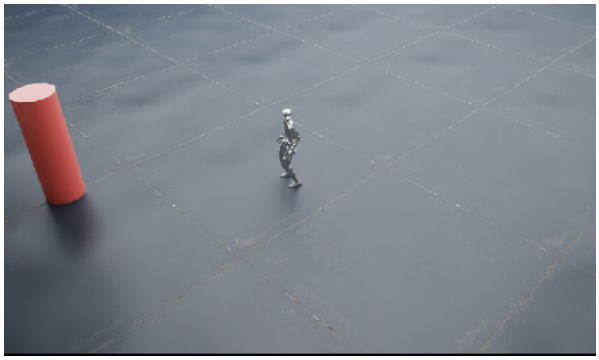
Σ_z is dominated by centroid wander, not ranging error, a different slice of the same cluster is visible frame to frame, which moves its centroid far more than any single depth return does, so it is measured from the sensor rather than assumed, and association gates on how far a track could plausibly have moved since its last hit rather than on nearest-neighbour alone, which is what keeps a fast cluster from being dropped and restarted as a new track. There is deliberately no manoeuvre model: an object seen for a few seconds through a narrow field of view does not supply enough evidence to fit anything richer than a constant velocity without fitting the noise instead. The filter is also, deliberately, not *extended*: the transition and the position measurement are both linear in o_j, so unlike the pose estimator of Section 2.1, which fuses orientation and is genuinely nonlinear, the ordinary Kalman filter already suffices here [3].
The estimate enters (24) by replacing the static p_j with a per-stage prediction,

$$p_{j,k} \;=\; \hat{p}_j + k\,\Delta t\,\hat{v}_j, \qquad k = 0, \dots, N, \tag{28}$$

so the penalty is evaluated against where the obstacle is expected to be at each node rather than where it was measured at the start of the cycle. This costs nothing in the structure the rest of the chapter relies on: M and N are still fixed at build time, so the symbolic graph and its sparsity pattern (Section 4.4) are untouched by (28); only the parameter buffer grows, from the $2M$ obstacle slots of Section 4.4 to $2M(N+1)$, written before every solve instead of once. One caution: (28) propagates the mean and drops Σ_j, and position uncertainty grows with τ^3 under (26), so a prediction six seconds out is markedly less trustworthy than one at the first node. Growing r_{obs} with the predicted node to compensate is the natural extension and is not implemented; the deployed profile does not yet track moving obstacles this way at all.



(a) The dynamic obstacle, segmented



(b) The scene in simulation

Figure 8: The moving-obstacle scenario the filter of this section is built for. **(a)** the tracker's own view as the obstacle passes: the grey mask is the segmented dynamic obstacle itself, not the floor or the wall, and its centroid is the measurement z_j that feeds (27); sparse coloured points are individual LiDAR/depth returns. **(b)** the corresponding Isaac Sim scene: the person-sized walking figure together with a cylinder, both synthetic dynamic obstacles inserted into the simulation to give the tracker something to segment, since nothing in the stock warehouse world moves on its own.

3.4.4 The obstacle as a true constraint: exact ℓ^1 penalty

The alternative to a penalty in the cost is a constraint with a slack. In such case, the obstacle becomes:

$$d(x_k, p_j) \;\geq\; d_{\text{safe}} - s_{jk}, \qquad s_{jk} \geq 0, \qquad k = 1, \ldots, N, \tag{29}$$

with the slack penalised in the cost either linearly ($\rho_s \sum s$) or quadratically ($\rho_s \sum s^2$). The question this settles is the one Section 3.4.2 could only raise: at what weight, if any, does the relaxation stop being a relaxation, so when the soft constraint becomes a hard one.

Penalising the slack in ℓ^1 makes the penalty *exact* [6]: above a finite threshold ($\mu^\star$, the norm of the multipliers of (29) at the solution), the penalised and the constrained problem share their minimisers and the slack vanishes identically, so beyond that weight the relaxation is no longer one. On the deployed problem the slack is already identically zero at $\rho_s = 1 \times 10^5$. Penalising in ℓ^2 never gets there at all: the slack decays as $1/\rho_s$ and the constraint is violated, however slightly, at every weight.

That answer is also the reason the obstacle is left in the cost. A constraint that is genuinely hard can be genuinely infeasible, and it becomes so exactly when the robot is already inside d_{safe}, the one situation in which a loop with no fallback trajectory must still return a command. This is consequence *(i)* of Section 3.4.2 read in the other direction: what the penalty gives up in guarantees it buys back in always having an answer. Below the threshold nothing is gained either, since the constraint is soft again and a soft constraint carrying one slack per node and per obstacle basically is (24) with more variables.

What it would cost to deploy. Switching the obstacle from the cost to the constraints takes the inequalities from 60 to 300 and the iteration count up by a factor of several. The deployed iteration cap would not be sufficient to handle the increased complexity. This is also what makes the solver comparison of Section 4.2.3 possible at all: the same system can be placed in two regimes that differ by a factor five in the number of inequalities, which is exactly the axis along which the standard advice on solver choice is stated.

3.4.5 Constraints

The deployed program carries only equality constraints and input bounds:

$$x_{k+1} - f(x_k, u_k) = 0, \qquad k = 0, \ldots, N-1, \tag{30a}$$

$$x_0 - \hat{x} = 0, \tag{30b}$$

$$v_{x,\min} \;\leq\; u_{1,k} \;\leq\; v_{x,\max}, \qquad k = 0, \ldots, N-1, \tag{30c}$$

$$|u_{2,k}| \leq v_{y,\max}, \qquad |u_{3,k}| \leq \omega_{\max}, \qquad k = 0, \ldots, N-1, \tag{30d}$$

with $v_{x,\min} = -0.20$ m/s, $v_{x,\max} = 0.40$ m/s, $v_{y,\max} = 0.20$ m/s and $\omega_{\max} = 0.40$ rad/s (Table 1). Three remarks:

- **No state constraints.** Neither the obstacle clearance nor the velocity states are constrained; the former is penalised, the latter are bounded implicitly because (19) is a convex combination of a bounded command and the previous velocity, so $v_{x,k}$ stays inside the input box for all k whenever $v_{x,0}$ is, which the state estimate does not guarantee, although on the deployed profile the degenerate lag of Section 3.3 collapses (19) to $v_{k+1} = u_k$ and the question does not survive the first node. The feasible set of the NLP is therefore a box intersected with an affine-in-X manifold, and a feasible point is always trivially available (integrate any admissible input sequence).
- **The input box is asymmetric.** Constraint (30c) admits reversing, but at half the forward speed, and (30d) caps strafing at the same fraction. Neither asymmetry is a safety derating applied to a symmetric machine: the lateral one is kinematic, since a biped strafes genuinely more slowly than it walks forward, while the backward one keeps the optimizer from adopting a retreat as a cheap shortcut at every stop. It is not free, however (pricing the retreat above the wide forward arc is one of the three structural facts behind the livelock of Section 5.4.1), and on hardware the bound may be tightened further to $v_{x,\min} = 0$, since the LiDAR sees poorly behind the robot.
- **Bounds are parameters, not constants.** The three velocity limits enter the NLP as CasADi parameters, so the supervisor can shrink them at runtime (Section 4.4) without rebuilding the problem, to slow the robot down in a crowded area, for instance, or to open the envelope again in free space.

3.4.6 Terminal ingredients and recursive feasibility

The deployed formulation carries no terminal set and no terminal constraint, so none of the standard guarantees of nominal stability and recursive feasibility [4, 13] applies. Recursive feasibility is nevertheless *structurally* present, for the reason given in Section 3.4.5: with no state constraints, every input sequence in the box is admissible, so the NLP always has a solution and the closed loop can never be aborted by infeasibility. What is lost is the guarantee that the closed loop converges: the tracking term can decrease along a trajectory that walks into a well of the obstacle penalty, and nothing but that penalty stops it.

It is worth saying plainly what that gap is. The controller plans 5.25 s ahead, executes the first step of the plan, and then starts again from scratch. Nothing ties one plan to the next, so a sequence of individually optimal plans need not go anywhere: each is the best available move inside a window that never looked past its own edge. The standard device for closing the gap is a condition on the last predicted state, to end every plan in a configuration that can be held indefinitely, because the previous plan, shifted forward by one step, is then always still executable. For a walking robot that configuration is standing still, balance being the business of the policy of Section 2.3.2, so the condition reads $v(N) = 0$: there must exist a braking trajectory inside the horizon.

On the plant used in this report that condition holds without being imposed. Section 2.5 sets $\tau_{v_x} = \tau_{v_y} = \tau_\omega = 0.00100$ s against $\Delta t = 0.35$ s, so every discrete lag coefficient is 1.000000 and the velocity update collapses to $v_{k+1} = u_k$: the model reaches zero velocity in a single sampling step, from any speed. A braking trajectory therefore always exists and the guarantee comes for free, for a reason that belongs to the simulation and not to the robot.

Supplying it explicitly would cost very little. The constraint is imposed softly, as

$$v_i(N) \;\le\; s_i, \qquad -v_i(N) \;\le\; s_i, \qquad s_i \;\ge\; 0, \qquad i \in \{x, y, \omega\}, \tag{31}$$

with $\rho_s \sum_i s_i$ added to the cost: three slack variables and nine inequalities, against the 141 variables and 156 constraints the program already carries, and all of them added once rather than once per node. Penalising in ℓ^1 is what makes the soft form acceptable; by the result of Section 3.4.4 the constraint is effectively hard wherever it is satisfiable and yields instead of aborting the loop where it is not.

Performing some measurements on a recorded mission, the terminal slack never exceeds zero, which is what the degeneracy above predicts, and the optimal objective rises by between 2.2 and 7.0.The absolute cost is small, roughly constant, and easy to account for; the plan has to predict its own braking, and the braking is discarded at the next cycle anyway.

It is worth asking what the terminal weight would have to be to carry a stability argument, because the answer is available in closed form. Linearising (19)–(21) about straight-line cruise at $v_{\text{ref}} = 0.27$ m/s and $\psi = 0$ gives a time-invariant pair (A, B), and with the deployed Q, R the discrete algebraic Riccati equation

$$P_\infty = A^\top P_\infty A - A^\top P_\infty B \left(R + B^\top P_\infty B\right)^{-1} B^\top P_\infty A + Q \tag{32}$$

returns the infinite-horizon cost-to-go $V_\infty(e) = e^\top P_\infty e$ and the gain $K_\infty = -(R + B^\top P_\infty B)^{-1} B^\top P_\infty A$; because Q is only semi-definite it is solved on $Q + 10^{-9} I$, the regularisation Section 3.4.1 anticipated. Two numbers follow. The deployed $Q_T = \rho_T Q = \text{diag}(20000, 20000, 400)$ over-weights the true cost-to-go $\text{diag}(P_\infty) =$

(207.9, 240.1, 6.3) by factors of 96, 83 and 63, and being diagonal it cannot represent the coupling $P_\infty[p_y, \psi] = 2.7$ that the linearisation produces. And the closed loop under K_∞ has spectral radius 0.31, so its transient decays by three orders of magnitude in about 2.1 s against a horizon of 5.25 s.
The second number explains the first. The horizon outlives the transient and the reference is regenerated every second, so the terminal term is a correction rather than the stabilising ingredient it would be in a short-horizon design; ρ_T is free to be mis-scaled by two orders of magnitude precisely because nothing depends on it, which is what the closed-loop sweep of Section 5.5 independently reports. This is the honest statement of what the deployed formulation has: a terminal *weight* that is a tuning parameter, and no terminal *ingredient* in the sense of [4, 13]. The pair (A, B) above is derived from the model as written here and was not read back from the CasADi graph, unlike the transition checked in Section 3.3.
Where the constraint would earn its place is on an actuator that is not instantaneous. Repeating a cycle with τ raised to 2 s, so that the lag coefficient falls to 0.16, produces the only strictly positive terminal slack in this study: the horizon is no longer long enough to stop. On hardware, where τ is fixed by the locomotion policy and the gait rather than chosen, $v(N) = 0$ inside a fixed horizon is precisely the condition that ties the cruise speed to the horizon length. This is the job currently done reactively, and outside the optimizer, by the adaptive velocity ceiling of Section 4.4.

3.4.7 On the absence of integral action

The formulation has no integral action and no disturbance estimator: the state (11) is not augmented with an input-disturbance model, and the reference is tracked in a purely proportional–predictive sense. Any constant unmodelled input, a lateral drift of the gait, a slope, a bias in the commanded twist, therefore produces a standing offset from the reference instead of being integrated away, and no choice of Q removes it. Two properties of the architecture are what make that acceptable here, and both are consequences of decisions taken for other reasons.
The offset does not accumulate, because the reference is regenerated from the current pose. The $A^\star$ layer of Section 3.2 replans at 1 Hz from where the robot *is*, not from where the previous plan said it would be. A standing offset is therefore absorbed into the next reference rather than integrated along the mission: it perturbs the route slightly and is gone at the following cycle. This is the same volatility that Section 5.4.1 shows to be a liability when the target rule is memoryless; here it is what keeps a constant disturbance from compounding.
The MPC is not the regulator. By Section 2.6 the tracker does not apply $u_0^\star$: it publishes a look-ahead point on the predicted trajectory, and a proportional law converts that point into the twist sent to the locomotion layer at 20 Hz. The loop that actually meets the disturbance is that one, so an integrator inside the NLP would correct a quantity that is not the one applied to the robot.
The standard remedy remains available and is worth stating precisely, because it is the natural continuation rather than an oversight. Augmenting the state with an integrated tracking error and optimizing the input increment Δu_k instead of u_k, the velocity-form MPC of [15, 18], restores offset-free tracking, at the price of carrying u_{k-1} in the state. That price is not only dimensional: Section 3.4.1 omits the rate penalty between the previously applied command and u_0 precisely so that the program does not inherit a memory of its own past output, and the velocity form would reintroduce exactly that memory. The two choices would have to be revisited together, which is why it is left as future work rather than added to the deployed program.

4. Optimization methodology

4.1. Problem dimensions and structure

Table 5 reports the size of the NLP at the deployed setting $N = 15$, $M = 8$. It is a medium-scale, sparse, smooth, non-convex problem: 141 decision variables, 96 equality constraints, 60 simple input bounds, and $2M(N + 1) = 256$ barrier terms in the objective; there are no obstacle *constraints*, so there is no obstacle multiplier and no non-trivial active set to identify: the entire inequality budget of the deployed program is the 60 box bounds on the inputs. A further 121 quantities enter as CasADi parameters: the measured state, the reference, the sentinel-padded obstacle positions and the three velocity bounds.
The Jacobian of (30a) is block bi-diagonal, with 6×9 blocks coupling (x_k, u_k) to x_{k+1}; the Hessian of the Lagrangian is block tri-diagonal, the off-diagonal input blocks coming from the rate penalty. Nothing in the problem couples distant stages, so the linear-algebra cost of one interior-point iteration is linear in N; at the deployed size the constraint Jacobian is 1.73% dense and the Hessian 4.45%, and Table 6 shows both densities falling as N grows while the nonzero counts grow linearly.

Table 5: NLP dimensions at the deployed configuration $N = 15$, $\Delta t = 0.35$ s, $M = 8$.

Quantity	Expression	Value
State variables	$6(N+1)$	96
Input variables	$3N$	45
Total decision variables	$6(N+1)+3N$	**141**
Dynamic equalities	$6N$	90
Initial-state equalities	6	6
Total equality constraints	$6(N+1)$	**96**
Input bounds	$4N$	**60**
Barrier terms (sigmoid + quadratic)	$2M(N+1)$	256

Table 6: Size and sparsity of the NLP against the prediction horizon, from the deployed profile. Bold: the deployed configuration.

N	vars	eq.	bounds	params	jac nnz	jac dens. [%]	hess nnz	hess dens. [%]
10	96	66	40	91	256	2.52	600	6.51
15	**141**	**96**	**60**	**121**	**381**	**1.73**	**885**	**4.45**
25	231	156	100	181	631	1.07	1455	2.73
50	456	306	200	331	1256	0.54	2880	1.39

4.2. Solver methodology

4.2.1 Overview

The NLP is solved with IPOPT [22], a primal–dual interior-point method with a filter line search. Writing the problem as $\min_z \phi(z)$ subject to $c(z) = 0$ and $z^L \le z \le z^U$, the bounds are replaced by a logarithmic barrier

$$\varphi_{\mu_b}(z) \;=\; \phi(z) \;-\; \mu_b \sum_i \Big[\ln\big(z_i - z_i^L\big) + \ln\big(z_i^U - z_i\big)\Big], \tag{33}$$

the resulting equality-constrained problem is solved approximately by a Newton step on the perturbed Karush–Kuhn–Tucker (KKT) system

$$\begin{bmatrix} \nabla^2_{zz}\mathcal{L} + \Sigma & \nabla c(z)^\top \\ \nabla c(z) & 0 \end{bmatrix} \begin{bmatrix} \Delta z \\ \Delta\lambda \end{bmatrix} = - \begin{bmatrix} \nabla\varphi_{\mu_b} \\ c(z) \end{bmatrix}, \tag{34}$$

and μ_b is driven to zero. Each iteration requires one factorisation of the sparse symmetric indefinite matrix in (34), performed by MUMPS on the pattern of Section 4.1. The Newton step therefore needs exact first and second derivatives at every iteration: the gradient of ϕ, the Jacobian ∇c and the Hessian of the Lagrangian $\nabla^2_{zz}\mathcal{L}$. None of them is coded by hand or approximated numerically. All three are produced by CasADi through algorithmic differentiation (AD), which walks the expression graph of Section 4.1 and composes the derivatives of the elementary operations exactly, to machine precision. Section 4.2.2 measures what this costs and what the alternatives would cost; Section 4.2.3 asks whether the interior-point method assumed here is the right family of solver for this problem in the first place.

4.2.2 Derivative computation

Every quantity the solver needs, like the gradient of the objective, the Jacobian of the constraints, the Hessian of the Lagrangian, is generated symbolically by CasADi and evaluated by generated code. The alternative is to employ other differentiation methods, such as finite differences. A comparison is done between the two approaches in order to assess if the second-order method above is affordable, computationally speaking.
At a representative solve the objective has 141 variables. One evaluation costs 132.4 μs and one full gradient by reverse-mode algorithmic differentiation (AD) 193.7 μs, i.e. 1.46 objective evaluations; it is small and constant, the important point is that it doesn't grow with the number of variables. The same gradient by finite differences costs 142 evaluations for the forward case and 282 for the central one, both growing linearly with n and is less accurate at every step size: the best relative error attainable is 1.27×10^{-7} in the first case and 9.84×10^{-11} in the second one, against machine precision for AD. The measured optimal steps coincide with the theoretical $\sqrt{\varepsilon_M}$ and $\varepsilon_M^{1/3}$, ε_M being the machine epsilon.

Table 7: Accuracy of the finite-difference gradient against the step size, measured against the AD gradient. The optimum sits at $h = 1.49 \times 10^{-8}$ forward (theory: $\sqrt{\varepsilon_M} = 1.49 \times 10^{-8}$) and $h = 6.06 \times 10^{-6}$ central (theory: $\varepsilon_M^{1/3} = 6.06 \times 10^{-6}$).

step h	rel. error, forward	rel. error, central
1.00×10^{-4}	9.18×10^{-4}	2.15×10^{-8}
6.06×10^{-6}	5.56×10^{-5}	9.84×10^{-11}
1.00×10^{-6}	9.17×10^{-6}	5.29×10^{-10}
1.49×10^{-8}	1.27×10^{-7}	3.77×10^{-8}
1.00×10^{-10}	9.66×10^{-6}	4.36×10^{-6}

For a real-time loop the cost settles the question: central differences alone would consume 30% of the 125 ms cycle budget, for a gradient that is worse, and the fraction grows linearly with the horizon. The ratio itself is a micro-benchmark on $\sim$100 μs timings and its second digit is not meaningful; what is stable, and is the point, is that it remains a small constant.

Exact Hessian against BFGS. Because AD supplies exact second derivatives at a cost comparable to the first, the full Newton system can be used rather than a quasi-Newton approximation [6].

Table 8: Interior-point iterations with the exact Hessian and with the limited-memory quasi-Newton approximation (L-BFGS), on the same instance. Bold: the lower iteration count.

Hessian	iterations	solve [ms]	$J^\star$	status
exact Hessian	**35**	328.4	3646	`Solve_Succeeded`
L-BFGS	89	1078.0	3646	`Solve_Succeeded`

On the same instance the exact Hessian converges in 35 iterations against 89 for the limited-memory quasi-Newton approximation, which means a 61% reduction, and both reach the same objective value. The total time is closer than the iteration counts suggest, because each quasi-Newton iteration is cheaper: this is the Newton / quasi-Newton trade-off.

4.2.3 Interior point and active set comparison

The standard advice is that active-set methods are better when the inequalities are few, while interior-point methods are better when they are many [6]. Although that advice is stated for convex programs, and this one is not, a comparison between the two approaches is conducted in this section, to assess which one is better in this specific setting. It is possible to move between the two regimes without changing anything else, by performing the same experiment with two different configurations: with the obstacles in the cost the program carries 60 inequalities and with the obstacles as genuine constraints (Section 3.4.4) it carries 300. That makes the advice testable rather than quotable.

Table 9: The same three instances solved by a primal–dual interior-point method and by an SQP with an active-set QP solver, in both obstacle regimes: *penalty* keeps the obstacles in the cost, ℓ^1 carries them as constraints with slacks (Section 3.4.4). Both solvers are started cold. Bold: the faster of the two. *fail* marks a run in which the active-set QP subproblem could not be solved at all. †the only large-regime run that returns a solution ends at an objective 0.2% away from IPOPT's, so that ratio is indicative rather than an exact like-for-like comparison.

regime	instance	ineq.	IPOPT ms / iter	SQP+qpOASES ms / iter	speed-up
penalty (obstacles in the cost)	bag ciclo 436	60	**67** / 20	146 / 6	2.2×
penalty (obstacles in the cost)	centred_pillar	60	**89** / 23	*fail* (860 ms)	—
penalty (obstacles in the cost)	narrow_gap	60	**225** / 77	275 / 7	1.2×
ℓ^1 (obstacles as constraints)	bag ciclo 436	300	**146** / 35	9307 / 16	63.6×
ℓ^1 (obstacles as constraints)	centred_pillar	300	**173** / 46	*fail* (13.2 s)	—
ℓ^1 (obstacles as constraints)	narrow_gap	300	**162** / 45	*fail* (11.1 s)	—

The rule does not reproduce: the active-set solver does not win even in the small regime, where the interior-point method is already between 1.2× and 2.2× faster on the two instances both methods solve. The *direction* is confirmed, indeed the margin widens to 63.6× when the inequalities multiply; so the break-even point, if it exists, lies below the smallest regime into which this problem can be put.
The failures matter more than the margins. In three of the six runs the active-set QP subproblem could not be solved at all, and the SQP returned nothing after 0.9 to 13 s, two of those three in the large regime, where it fails on two instances out of three. So the comparison is not that one method is slower than the other: on this program the interior-point method returns a certified solution every time, and the active-set route returns one in half the cases.

4.3. Optimality conditions on the deployed problem

Everything above assumes that what the solver returns is a solution. That is a verifiable statement, and this section verifies it on cycles extracted from a recorded mission rather than on a synthetic instance. The cycles are sampled across the 201 moving cycles of Section 5.1, spanning the whole mission from the first metre to the arrival, so the diagnostics below describe a trajectory through the operating envelope and not one favourable point in it.

Table 10: Optimality diagnostics at sampled cycles of the recorded mission. "active" counts equalities and active bounds together; "rank" is the rank of the Jacobian of the active constraints, so LICQ holds when the two coincide.

cycle	t [s]	active	of which bounds	rank	LICQ	cone dim.	$\lambda_{\min}$ proj.
0	0	103	7	103	yes	38	1.33×10^{0}
86	46	101	5	101	yes	40	1.26×10^{0}
173	85	98	2	98	yes	43	1.25×10^{0}
260	120	101	5	101	yes	40	1.31×10^{0}
347	153	97	1	97	yes	44	1.24×10^{0}
433	191	100	4	100	yes	41	1.26×10^{0}
520	245	100	4	100	yes	41	1.25×10^{0}
607	315	101	5	101	yes	40	1.28×10^{0}
694	358	98	2	98	yes	43	1.23×10^{0}

At 9 sampled cycles, the linear independence constraint qualification (LICQ) [6] holds at every one (yes), the rank of the Jacobian of the active constraints equals their number; so the KKT multipliers exist and are *unique*. Strict complementarity holds at every one (yes), which means the critical cone coincides with the null space of the active Jacobian and the second-order condition can be checked exactly rather than conservatively. And the reduced Hessian is positive definite on that cone at every cycle (yes), the smallest projected eigenvalue over the profile being 1.23×10^{0}. Taken together these are a certificate of strict local optimality, which is the most that can be claimed for a non-convex program.

Shape of the active set. The critical cone holds between 38 and 44 dimensions across the sampled cycles and does not collapse: with 141 variables and 98 active constraints at the end, the freedom the controller has when it arrives is the freedom it had when it started. Almost every active row is an equality of the dynamics; the input bounds contribute a handful per cycle and never more. The controller is cost-driven for the whole mission.

4.4. Implementation and configuration

The program is built once, symbolically, with the CasADi `Opti` interface, and every quantity that changes between cycles is a *parameter* rather than a constant:

- $\hat{x} \in \mathbb{R}^6$, the measured initial state;
- $X_{\text{ref}} \in \mathbb{R}^{6\times(N+1)}$, the reference tracked by the Q block of (23);
- $\mathcal{O} \in \mathbb{R}^{2\times M}$, the (sentinel-padded) obstacle positions;
- the three velocity bounds of (30c)–(30d).

At the deployed $N = 15$ and $M = 8$ these account for $6 + 96 + 16 + 3 = 121$ parameter slots. Consequently the expression graph, the derivative code and the sparsity patterns are generated exactly once at start-up; a cycle only writes numbers into parameter slots and calls the solver. The graph is flattened with `expand: true`, which

converts the `MX` graph into scalar `SX` operations and removes the interpreter overhead that would otherwise dominate a problem whose objective and constraints together expand to 3603 scalar operations. The first solve after start-up therefore pays a one-off construction and code-generation cost, 0.5 s at the deployed configuration and up to 0.6 s at the longest horizon tested (Section 5.3); this is the single deadline violation of the entire run, and the deployment absorbs it by holding the robot still during initialisation.

Solver options are deliberately austere: `max_iter`= 40, `print_level`= 0, `warm_start_init_point` enabled. No CPU-time limit is imposed inside IPOPT; the real-time protection lives in the supervisor instead, which is the more robust arrangement because it keeps the solver deterministic. A second reason is specific to the architecture of Section 2.1: a non-converged iterate is still usable as a reference by the downstream setpoint stage, which would not hold if $u_0^\star$ were applied to the joints directly. Accepting an inexact solution by design rather than as a fallback is what the real-time iteration scheme does [5]. The supervisor implements three mechanisms, all part of the deployed configuration and all of which exist because a loop at 125 ms per cycle cannot afford to wait for a perfect solution:

1. **Warm-start hygiene**, the cache is cleared on solver failure and on a cost spike (Section 4.2.3).
2. **Zero-velocity fallback**, after three consecutive IPOPT failures the tracker returns a zero input, which is always feasible and always safe; this is preferable to applying the last usable solution, which is by then stale. The fallback is deliberately transient: it skips a single cycle and then resets its own counter, because a latching fallback would leave a robot that never recovers.
3. **Adaptive input bounds**, when the observed failure rate exceeds 30% the supervisor shrinks $v_{x,\max}$ by 10% through the parameter interface, reducing the distance the horizon covers and thereby the number of active barrier terms, and walks it back up in 5% steps once the failure rate drops below 5%.

Unlike the mechanisms of a system that never exercises them, these do fire. Over the 8688 solves of the extended horizon campaign the solver failed 308 times, and the failures are strongly graded by horizon length: 0.7% at $N = 15$, 3.5% at $N = 26$ and 6.5% at $N = 40$. This is worth stating plainly, because it is the honest form of the argument for the deployed horizon: the long horizons are not merely slower, they are the ones on which a bounded iteration count stops being enough, and the fallback rather than the optimum decides the command.

Table 11: The deployed profile `planner_params_g1.yaml` in full. Everything the optimization program reads is here: the prediction model and the horizon, the admissible input set $\mathcal{U}$, the eleven cost weights that make up θ in (35), the discrete layer of Section 3.2 they are tuned against, and the solver options. The weights are the hand-set values actually shipped, not the $\hat{\theta}$ of Section 4.5, for the reason given in Section 5.5.

Prediction model and horizon			*Cost weights θ*		
Prediction horizon	N	15	Position	Q_x	200.0
Prediction step	Δt [s]	0.35		Q_y	200.0
Look-ahead	$N\Delta t$ [s]	5.25	Heading	Q_ψ	4.0
Re-solve rate	[Hz]	8	Terminal scale	ρ_T	100.0
Actuator lag	τ [s]	0.00100	Forward effort	R_{v_x}	1.0
Integrator	,	mid-point	Lateral effort	R_{v_y}	6.0
Obstacle terms	M	8	Yaw effort	R_ω	0.4
Obstacle search radius	[m]	2.0	Input rate	R_{jerk}	0.4
Admissible input set $\mathcal{U}$			Barrier height	W_{obs}	120
Forward speed	v_x [m/s]	$[-0.20, 0.40]$	Barrier steepness	α_{obs}	6.00
Lateral speed	$\lvert v_y \rvert$ [m/s]	≤ 0.20	Safety radius	r_{obs} [m]	0.40
Yaw rate	$\lvert \omega \rvert$ [rad/s]	≤ 0.40			
Cruise speed	v_{ref} [m/s]	0.27	*Solver (IPOPT)*		
Grid and A* layer			Max iterations	,	40
Resolution	δ [m]	0.20	Warm start	,	on
Half-width	h_w [m]	6.0	Print level	,	0
Inflation	σ [m]	0.31	Graph	`expand`	true
Hard threshold	P_{thr}	0.10			
Edge weight	w_{obs}	35.0			
Replan rate	[Hz]	1.0			

4.5. Offline weight tuning by Bayesian optimization

The cost (23) and the obstacle term (24) are parametrised by eleven weights,

$$\theta = \big(Q_x,\, Q_y,\, Q_\psi,\, \rho_T,\, R_{v_x},\, R_{v_y},\, R_\omega,\, R_{\text{jerk}},\, W_{\text{obs}},\, \alpha_{\text{obs}},\, r_{\text{obs}}\big) \in \Theta \subset \mathbb{R}^{11}, \tag{35}$$

which are not physical quantities but the decision variables of a second-level problem

$$\hat{\theta} \;=\; \arg\min_{\theta\in\Theta}\; \mathcal{J}(\theta), \tag{36}$$

where $\mathcal{J}$ measures closed-loop navigation quality. This is the third optimization problem in the stack and the only one that is not solved online. It shares none of the structure that Sections 4.1 and 4.2.2 exploit: $\mathcal{J}$ has no closed form, no derivatives, and one evaluation costs a full navigation rollout. It is a black-box problem, and the standard tool is a Gaussian-process surrogate [17, 21]. Nothing below is specific to the platform: the scheme sees only a vector of weights and a scalar score, so it applies unchanged to any robot the tracker of Section 3.4 drives.

The obstacle is dimension, and it dictates the method. A surrogate over $\Theta \subset \mathbb{R}^{11}$ needs a sample density that a rollout-priced objective cannot supply: a budget of n evaluations covers each axis with $n^{1/11}$ points, so even a hundred rollouts amount to fewer than two points per coordinate.

The objective. Each evaluation is a full closed-loop mission driving the production tracker, with a time cap:

$$\mathcal{J}(\theta) = \begin{cases} T_{\text{goal}}/T_{\text{ref}}, & \text{goal reached,} \\ 10 + 5\, d_{\text{final}}/d_0, & \text{otherwise,} \end{cases} \;+\; c_u\, \overline{\|\Delta u\|} \;+\; c_t\, \bar{t} \;+\; c_d \max\big(0,\, d_{\text{safe}} - d_{\min}\big), \tag{37}$$

with $T_{\text{ref}} = d_0/v_{\text{ref}}$ the time a straight run at cruise speed would take, d_{final} the residual distance to the goal, $\overline{\|\Delta u\|}$ the mean input increment, $\bar{t}$ the mean solve time and $d_{\min}$ the minimum clearance. The branch structure is the part that matters. A configuration that does not complete the mission is scored on the *fraction of the distance it closed*, so the objective stays informative on failures instead of saturating at a constant penalty; and because (37) is evaluated in closed loop rather than on a recorded trajectory, it can observe a deadlock, which is a failure mode no one-step prediction score can see.

Per-axis surrogates and coordinate descent. Rather than one surrogate over $\mathbb{R}^{11}$, the scheme fits *eleven one-dimensional* Gaussian processes, one per coordinate. Writing $\theta = (\theta_i, \theta_{-i})$, each axis is explored on a uniform design of m values spanning its interval while θ_{-i} is held fixed, and a 1-D GP with a Matérn-5/2 kernel and a white-noise term,

$$k(\theta_i, \theta_i') = \sigma_f^2\Big(1 + \sqrt{5}\,\tfrac{|\theta_i - \theta_i'|}{\ell} + \tfrac{5}{3}\,\tfrac{(\theta_i - \theta_i')^2}{\ell^2}\Big) e^{-\sqrt{5}|\theta_i - \theta_i'|/\ell} + \sigma_n^2 \delta(\theta_i, \theta_i'), \tag{38}$$

is fitted to those observations by maximising the log marginal likelihood. With $\Theta_i = (\theta_i^{(1)}, \ldots, \theta_i^{(m)})$ the design on axis i and y_i its m scores, the posterior mean and variance at a query point are

$$\mu_i(\theta_i) = \bar{y}_i + k_i(\theta_i)^\top K_i^{-1}\big(y_i - \bar{y}_i \mathbf{1}\big), \qquad \sigma_i^2(\theta_i) = k(\theta_i, \theta_i) - k_i(\theta_i)^\top K_i^{-1} k_i(\theta_i), \tag{39}$$

with $[k_i(\theta_i)]_a = k(\theta_i, \theta_i^{(a)})$, $[K_i]_{ab} = k(\theta_i^{(a)}, \theta_i^{(b)})$ and $\bar{y}_i$ the sample mean removed before fitting. The white-noise term of (38) reaches only the diagonal of K_i, which is what makes μ_i smooth the scatter instead of interpolating it. The axis optimum is the minimiser of the posterior *mean* over a fine grid,

$$\hat{\theta}_i \;=\; \arg\min_{\theta_i \in [\ell_i, u_i]}\; \mu_i(\theta_i), \tag{40}$$

which uses the smoothed estimate rather than the best sampled point and therefore does not require the optimum to fall on a grid node. The total cost is $11m + 1$ rollouts, and the by-product is worth as much as the optimum: eleven curves with error bands, each showing how much the objective actually depends on the weight in question.

Three decisions make the scheme defensible, and each is a place where it could quietly go wrong. *Separability.* Assembling per-axis optima into one vector is exact only when $\mathcal{J}(\theta) \approx \sum_i f_i(\theta_i)$, which is not true here: the barrier height and the safety radius interact, since a tall barrier is harmless when its radius is small. The assumption is therefore repaired by the next decision and verified a posteriori by evaluating the assembled vector. *Sequential update and ordering.* The axes are swept in decreasing order of influence and each optimum

is fixed before the next axis is visited, so later sweeps are conducted about an already improved operating point. This is coordinate descent [23] with a GP line search, and it is what makes the scheme robust to the interactions separability ignores: moving the dominant coordinate first takes the base point out of the region where the mission fails, so every subsequent sweep is measured where the objective discriminates. Influence is measured rather than assumed, and it costs no rollouts of its own, a GP fitted over all eleven coordinates at once, to the trials already on record, carries one length scale per coordinate, and a short length scale says the objective changes appreciably over a small displacement of that weight. The budget of $11m + 1$ rollouts should be read as a restriction: each axis is visited exactly once, with no second pass to repair a choice made about a base point that later sweeps have since moved. *Significance floor.* A 1-D GP fitted to a handful of points returns an optimum on any axis, including one on which the objective does not move. An axis is accepted only if the spread of its observations exceeds the run-to-run scatter; otherwise the weight keeps its default. Without that floor the procedure reports eleven confident optima and several of them are noise.

What the procedure can be expected to gain. A tuning procedure of this kind can be expected to benefit in accuracy and smoothness, and any large gain it reports must come from the failure branch of (37) (from configurations that do not reach the goal at all) rather than from trimming one that already works. That is a useful prediction to hold in advance of a campaign, because it says which number to distrust.

5. Results and solver performance

5.1. Experimental setup

All results in this section are produced by an offline harness that imports the *production* modules (the grid, the $A^\star$ planner and the MPC tracker), so every number is generated by the deployed formulation and the deployed solver, not by a re-implementation. Two kinds of experiment are used and they answer different questions.
Replay from recorded missions. The recorded data are a full run of the G1 in the industrial world: the poses, the LiDAR scans, the $A^\star$ paths and the MPC predictions as they were published. Replaying it gives real operating points, with the geometry and the clutter the robot actually met, and is what Section 4.3 uses. Its size is worth stating, because every replay figure is a sample from it rather than a single measurement: the mission spans 695 control cycles, 201 of them with the robot in motion and an $A^\star$ reference long enough to re-solve against. The analyses below draw from those 201, and each reports the spread over its sample rather than a representative cycle. What the replay cannot answer is counterfactual questions.
Closed-loop synthetic scenarios. For counterfactuals (what if the horizon were shorter, what if the weights were different) the harness runs full missions in scripted worlds, with a ray-marched planar LiDAR and the discrete model of Section 3.3 as the plant. This is what Section 5.3 uses. Three of these worlds recur: *narrow gap*, two obstacles with a gap between them; *U-trap*, a concavity open towards the robot; and *corridor*, a straight passage with a staggered pillar in it, used in the three-parameter sweep of Section 5.3.
A further family of worlds is used only in Section 5.4, because it probes a different layer: ten worlds, six of which place a *non-convex* obstacle between the robot and the goal and four of which only look as though they do. They are run with *limited perception*: the robot sees only out to the LiDAR range and only what is not occluded, and accumulates what it sees. That last detail is not a refinement but the whole experiment, because with the obstacle known in advance the discrete planner never enters the trap and the failure under study does not occur.

5.2. The cost landscape in state and decision space

Before the campaigns, it is worth looking at the object all of them are about. Fig. 9 draws, over the world plane of the recorded industrial run, the cost of *being* at a point,

$$c(p) \;=\; \underbrace{\text{tracking cost of the reference}}_{\text{the } Q \text{ block of Eq. (23)}} \;+\; \underbrace{J_{\text{obs}}(p)}_{\text{Eq. (24)}}, \tag{41}$$

with the executed trajectory, the $A^\star$ reference and one MPC horizon drawn on the surface, and the per-cycle optimal cost $J^\star$ underneath.
The distinction that figure makes visible is the one between c and J, and it is the reason the stack uses an optimizer rather than a reactive law. The quantity plotted is c, the cost of occupying a point; what the program minimises is

$$J(U) \;=\; \sum_{k=0}^{N} c(p_k) \;+\; \text{input terms}, \tag{42}$$

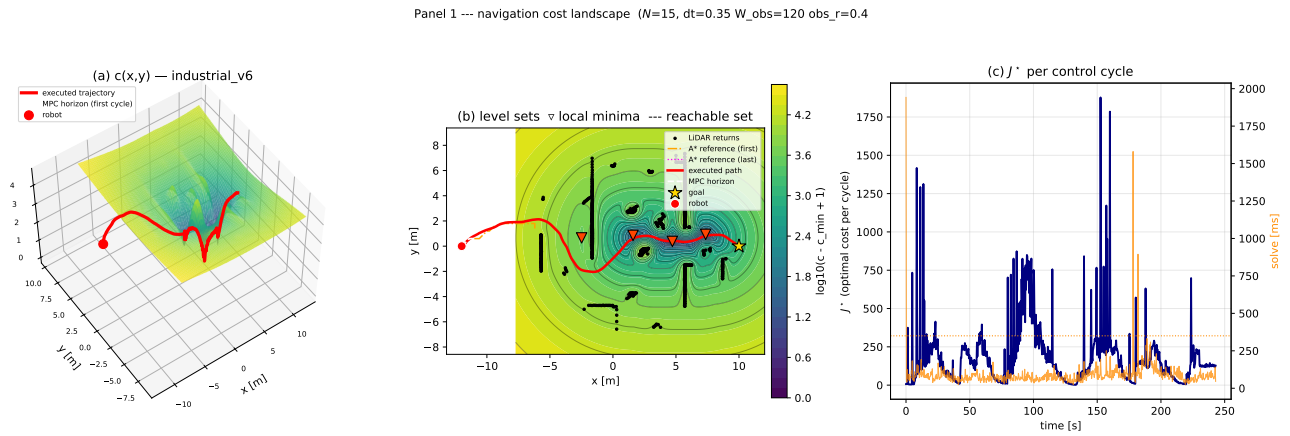


Figure 9: The navigation cost landscape of the recorded mission. **(a)** the surface $c(p)$ of Eq. (41) over the world plane, with the executed trajectory (red) and one MPC horizon (dashed) drawn *on* the surface: the obstacle term raises a ridge along every wall of the plant, the tracking term forms the valley along the $A^\star$ reference. **(b)** the same field as level sets, with the local minima marked and the shaded region showing the set reachable within one horizon under $\mathcal{U}$; the constraint, not the cost, is what excludes most of the plane. **(c)** the optimal cost $J^\star$ of each control cycle, and the solve time beside it. The point worth noticing is in panel (a): *the trajectory does not follow steepest descent*, and climbs the surface wherever doing so lowers the sum over the horizon.

the cost of a whole *trajectory*. A controller that descended c pointwise would be an artificial potential field, and it would stop at the first local minimum of the surface. The MPC minimises the sum along a horizon instead, so it can, and in panel (a) visibly does, move *uphill* on c when that buys a lower total. This is what lets the same cost function carry the robot out of a concave pocket, and it is the sense in which the obstacle term of Eq. (24) is a penalty inside an optimization problem rather than a repulsive force.
One further reading, which anticipates Section 4.3. The shaded region in panel (b) is not a level set of the cost: it is the set of points the robot can reach within one horizon given $\mathcal{U}$. Most of the plane is excluded from the decision by the *constraints*, not by the objective, and on this platform, with $v_{y,\max} = 0.20$ m/s against a forward bound of 0.40 m/s and reversing admitted only at a fraction of that, the region is markedly asymmetric about the heading. The cost is what chooses inside it; the input set is what draws it.

5.3. Horizon design: prediction horizon, sampling time and control horizon

Three parameters shape the program: $N\Delta t$ sets how far the controller sees, N sets how many variables it carries, and the control horizon N_c frees only the first N_c inputs and holds the rest fixed, shrinking the program without shortening the prediction. Δt is not chosen from plant bandwidth, Section 2.5 gives a first-order lag with no pole to oversample, but from the geometric inter-sample bound (25), which leaves a factor of three of margin at the deployed values. Two closed-loop campaigns settle the rest: one over $(N, \Delta t)$, one adding N_c.
Fig. 10 carries the first campaign, and two readings from it are what matter. *Safety is governed by the look-ahead $N\Delta t$ alone, with a sharp threshold near 3 s* (panel c): below it the controller cannot see an obstacle in time to react, and drives at the goal regardless of what is in the way, which panels (a)–(b) show costing nothing in *narrow gap* but a collision in *U-trap*; above it every configuration clears by a comfortable, flat margin no matter how N and Δt split the product. *Cost is governed by N alone* (panel d): solve time climbs with N and barely moves with Δt, so the cheapest way to reach a given look-ahead is a large Δt and a small N. The deployed $N\Delta t = 5.25$ s sits at roughly twice the safety threshold, and ranking every configuration on time, clearance and solve time together leaves it beaten by only one other point, and only marginally, nothing on this campaign is worth switching to.

Control horizon. The first campaign cannot separate the two roles of N, since looking further and carrying more variables are there the same move. A second campaign does, over 26 paired cases at fixed prediction horizon: cutting the degrees of freedom is *strictly* free, time and clearance unchanged while solve time falls, in only 1 of them; elsewhere clearance is what pays, and it recovers smoothly as N_c grows, from 0.301 m at $N_c = 1$ to the full 0.811 m by $N_c = 10$ at $N = 15$ in *narrow gap*. About ten free inputs is enough to negotiate the corner; the rest can be frozen almost for free.

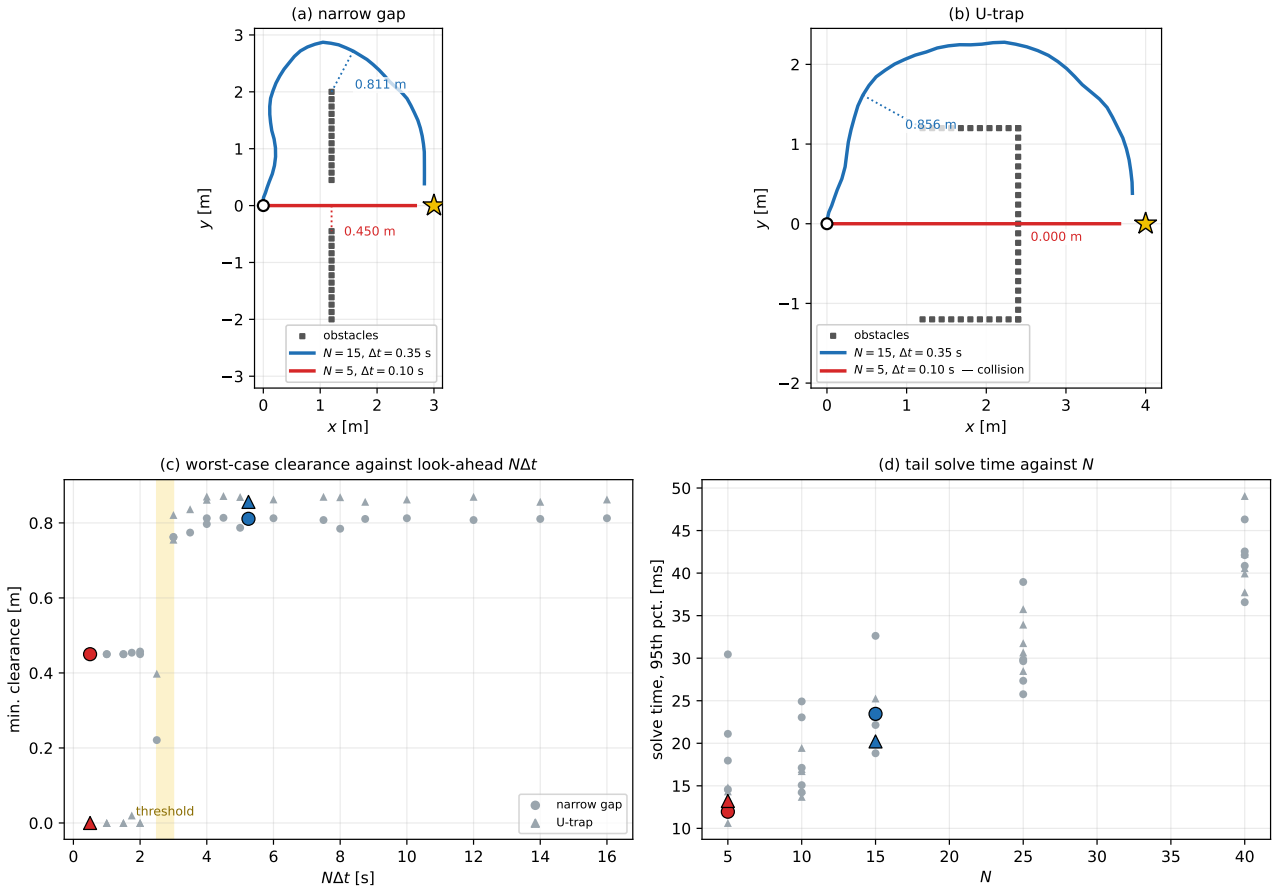


Figure 10: The deployed configuration against the cheapest one the campaign returns. **(a)**, **(b)** the executed trajectories in the two worlds, with the point of minimum clearance marked. At $N = 15$, $\Delta t = 0.35$ s the controller routes around the obstacle in both; at $N = 5$, $\Delta t = 0.10$ s it drives straight at the goal, which in *narrow gap* means taking the gap down its centreline and in *U-trap* means passing *through* the back wall. **(c)** minimum clearance against the look-ahead $N\Delta t$ for all 50 runs of the campaign: the transition is between 2.5 and 3 s and it does not depend on which $(N, \Delta t)$ pair realises the look-ahead. **(d)** the tail solve time against N, the only parameter that governs it. The two highlighted configurations are the same in all four panels.

The three parameters together. Sweeping N, Δt and N_c jointly over three worlds and aggregating conservatively (mean time, worst clearance, worst tail time), seven configurations dominate the deployed one. The best, $N = 26$, $\Delta t = 0.35$ s, $N_c = 10$, matches the deployed time to goal and clearance at 24.3 ms of tail solve time against 59.3, with 192 decision variables against 141: a longer look-ahead in a cheaper program. This is the report's cleanest evidence that the two horizons are separate design parameters, and it answers what the first campaign left open, a long horizon costs through its variable count, not through its length.

On move blocking. The deployed profile does not use blocking, and computation is the resource it is short of: on the recorded mission the 95th percentile solve time is 157.9 ms against a 125 ms cycle budget, so the loop misses its period on the hard cycles. The control horizon above, one free block followed by one long one, is the degenerate case of blocking and is exactly what recovers the period in the joint sweep; adopting it at $N_c = 10$ with $N = 26$ is the change this section supports making before the next recording campaign. One implementation detail matters: the input bounds must be imposed on the free columns only, not repeated over the blocked steps, or the duplicate rows violate LICQ and break the multiplier analysis of Section 4.3.

How far these numbers carry. These worlds are synthetic with static obstacles, so the clearance margin the look-ahead buys would matter more, not less, against moving obstacles or a slower discrete layer. Tail solve times are comparable only within one campaign, not across the two reported here.

5.4. Non-convex obstacles and the local-target rule

5.4.1 What is being measured

The clearest illustration in this study of the difference between solving an optimization problem well and posing the right one does not involve the NLP at all. With the projection rule (17) the stack livelocks in front of a non-convex obstacle: the ray to the goal enters the concavity, so the local target lands inside it, $A^\star$ correctly routes around the obstacle *inside the window*, the MPC tracks that path faithfully, the robot leaves, and the moment it is out the ray points back in. Every NLP in that run is solved to optimality, and every one of them answers a question that should not have been asked. The mechanism and the repair are in Section 3.2; measured here is whether the repair works, and at what price.

The design of the campaign matters as much as its outcome. A target rule that simply refused concavities would pass every trap and be useless, because most concave-looking scenes are traversable and the shortest route runs straight through them. The worlds are therefore in two groups. Six are genuine *traps*: a corridor closed at the far end, a horseshoe, an L with two nested closures, and three variants of a wall too long for its ends to fit in the window, with the gap on the north side, on the south side, and apparently north but in fact a dead end. Four are *controls* that look like traps to the sensor but are open: a corridor identical to the dead end but not closed, a wide corridor with three staggered baffles, a wall with a single 1.6 m door, and the warehouse of scattered convex obstacles. On the traps the geodesic rule must turn the robot around; on the controls it must do nothing at all.

Two settings of the harness decide what the numbers mean. The planner runs the non-convex overlay of the deployed profile, a 20×20 m window with the accumulated map not expiring, under the limited-perception model of Section 5.1: the trap is produced by the 8 m LiDAR range and not by the window, since unseen space is optimistically free either way. And the plant refuses to penetrate. This is a correction to the harness rather than a refinement of it: the integrator of Section 5.1 has no contact model, $A^\star$ treats obstacles as blocked cells and the MPC as a soft penalty, so nothing prevented the fallback proportional law from driving the robot through a wall. An earlier run of this campaign reported three goals reached with clearances of 0.039 to 0.049 m, which is not a result but a trajectory threading the 0.12 m sampling gap where two wall segments meet. Steps that would bring the robot within 0.25 m of an obstacle are now rejected, component by component so that the robot may still slide along a wall. The threshold sits above the sampling pitch and below the narrowest legitimate channel in the ten worlds, the 0.81 m of the door, so it closes no passage that exists.

5.4.2 Results

Every world is run once under the projection rule (17) and once under the deployed geodesic $\arg\min$ (16), with the per-world time budget of the simulator, for twenty missions in all. The point of the split is that a rule can only be judged on both halves at once: the six traps ask whether it turns the robot around, and the four controls ask whether it leaves alone the scenes that merely look closed to an 8 m sensor. A rule that passed only the first group would be a rule that refuses concavities, which is worse than the one it replaces. The two long-wall worlds are additionally re-run at seven spawn displacements each, so the branch the scene selects is measured rather than assumed. Figure 11 shows the geometry and both trajectories, Table 12 the numbers.

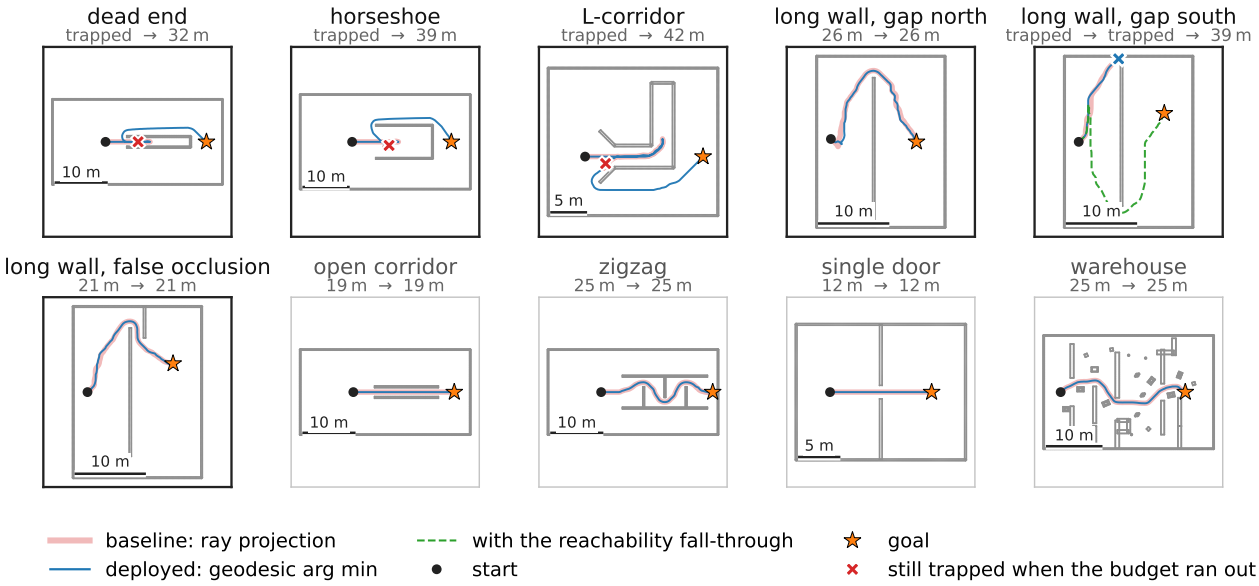


Figure 11: The ten worlds of the campaign: the wide pale trace is the baseline ray projection (17), the thin dark one the deployed geodesic arg min (16), both under limited perception. Heavy frames are the six *traps*, light frames the four *controls* that only look like traps; the subtitle gives the distance travelled by each rule. The baseline is drawn underneath and wider on purpose, because wherever the two rules choose the same side the trajectories coincide and one drawn over the other would simply hide it, and because on *dead end*, *horseshoe* and *L-corridor* the 79 to 153 m it travels are a limit cycle packed into a few metres at the mouth of the obstacle. In *long wall, gap north* both rules choose the open side and both arrive; in *long wall, gap south* the goal offset forces both onto the closed side and both stop there. The controls are what make the result a result: a rule that refused to enter concave-looking scenes would pass every trap and fail all four of these, where the correct behaviour is to drive straight through. The dashed trace appears in the *long wall, gap south* panel alone and is the reachability fall-through discussed in Section 5.4.2; on the other nine worlds it reproduces the deployed run exactly, so drawing it there would only thicken the line.

Table 12: Escape from non-convex obstacles under limited perception: the baseline ray projection (17) against the deployed geodesic arg min (16), one run each, with the per-world time budget of the simulator. Above the double rule the six traps, below it the four controls. Reversals are changes of sign of the longitudinal velocity, the signature of the limit cycle. Clearance is the closest approach to an obstacle over the whole run, against a body radius of 0.35 m; on the runs that fail it is the 0.25 m at which the plant stops the robot against the wall. Time is reported only for runs that reached the goal. The *long wall, gap north* row is taken at a spawn displaced 0.4 m along the wall: with the goal on the axis the side is chosen by numerical noise, so a single run there compares the two spawns and not the two rules, and the sweep that establishes this is reported below.

World	Target rule	Goal	Time [s]	Path [m]	Rev.	Clearance [m]
dead end	projection	no	,	79.4	140	0.659
	geodesic	yes	81.5	31.7	2	0.669
horseshoe	projection	no	,	93.4	117	1.919
	geodesic	yes	100.4	38.5	2	0.853
L-corridor	projection	no	,	153.1	300	0.801
	geodesic	yes	112.7	41.8	8	0.801
long wall, gap north	projection	yes	64.8	26.2	10	0.919
	geodesic	yes	64.8	26.3	10	0.891
long wall, gap south	projection	no	,	13.6	6	0.268
	geodesic	no	,	13.7	4	0.255
long wall, false occlusion	projection	yes	51.8	21.3	8	0.768
	geodesic	yes	52.5	21.5	8	0.786
open corridor	projection	yes	47.2	18.7	0	0.850
	geodesic	yes	47.2	18.7	0	0.850
zigzag	projection	yes	63.0	25.4	0	0.800
	geodesic	yes	63.0	25.4	0	0.813
single door	projection	yes	29.7	11.8	0	0.800
	geodesic	yes	29.7	11.8	0	0.800
warehouse	projection	yes	62.6	25.4	0	0.543
	geodesic	yes	62.6	25.4	0	0.543

The traps. The projection rule reaches the goal on two of the six. On *dead end*, *horseshoe* and *L-corridor* it exhausts the budget without escaping, with the signature Section 3.2 predicts: 140, 117 and 300 reversals over 79, 93 and 153 m of travel that never leave the mouth of the obstacle. That is a limit cycle, measured, not a slow traversal. On *long wall, gap south* it does not even oscillate; it drives into the wall and stays there, having travelled 14 m in a budget of 480 s.
The geodesic rule resolves five of the six, failing only on *long wall, gap south*. Clearances on those five run from 0.67 to 0.89 m against a body radius of 0.35 m, and the reversal count collapses from up to 300 to at most 10. On *dead end* and *horseshoe* the escape costs exactly two reversals: the robot enters, observes the closing wall, and leaves. That is the optimistic field correcting itself as the sensor discovers, and it is the direct measurement of the argument of Section 3.2. The cost is bounded: over the nine missions it completes, the path is a median of 1.7 times the straight-line distance and at worst 2.6 times, so the escape is a detour and not an exploration of the world.

The two long-wall worlds. The wall of *long wall, gap north* is longer than the LiDAR range, so from the start neither of its ends is observable and the robot cannot know which one is open. With the goal on the axis the two sides are symmetric, the choice falls to numerical noise, and the 2.0 m hysteresis then holds whichever side came out. This is a property of the scene and not of the rule, so it was measured rather than argued: the spawn was displaced along the wall in seven steps within ± 0.6 m and both rules were run at each. The projection rule chose north in two cases of seven and the geodesic rule in four, and neither choice is monotone in the displacement, which is what a coin flip looks like. What the two rules do agree on is the consequence: all six runs that chose north reached the goal in 26 to 27 m, and all eight that chose south were still pinned against the closed end when the budget expired. The side decides the mission and the target rule does not, so the row of Table 12 for this world is reported at a displacement where both rules happen to choose north, and

it should be read as an illustration of the lucky branch rather than as a comparison.
The second world, *long wall, gap south*, is the same wall with the coin flip removed. The gap is at the southern end and the goal is placed 4 m north of the axis, which makes the north side shorter on everything that can be observed from the start, so both rules take it, and it is the closed one. This is the branch the first world cannot show, and both rules follow the wall to its northern end and stop there.

Reachability of the selected target. The failure is not the metric. Interrogated at the pose where the run stalls, the field is intact and every free cell is reachable; the selected target is the boundary cell at (5.8, 3.84), whose geodesic distance to the goal is 0.40 m, which is the correct answer to the question the rule asks. The rule scores candidates by their distance *to the goal* and never by whether the robot can reach *them*: the chosen cell lies on the far side of a wall that spans the whole window, $A^\star$ finds no path to it, the historical fallback is the ray projection which points through the same wall, the planner publishes nothing, the previous reference stands, and the robot presses into the wall for the remaining 470 s. The geodesic field is global and $A^\star$ is windowed, and nothing in (16) reconciles the two.
The repair is correspondingly narrow: when the discrete search returns no path to the chosen candidate, walk down the ranking and take the first one it can actually reach. It was implemented behind a flag, off in the deployed profile, and the ten worlds were run again with it on. Nine of the ten reproduce the deployed run to every reported digit, which is the property a repair of this kind must have: it fires only in the cycles where the first candidate is unreachable, and those do not occur when the target is already sound. On *long wall, gap south* the robot now heads north as before, abandons that side as soon as the search cannot reach the target it has selected rather than after driving to the closed end, turns back and leaves through the southern gap, reaching the goal in 96.9 s over 38.8 m with 14 reversals and a clearance of 0.908 m. That is the dashed trace of Figure 11. Run on the three displacements of *long wall, gap north* at which the geodesic rule had chosen the closed side, it recovers all three, in 92 to 95 s over 36 to 37 m at clearances of 0.89 to 0.91 m. The wrong guess stops being fatal, which is the property the wall family was built to test, and the traps go to six out of six. The numbers in Table 12 remain those of the deployed configuration.

The controls. On all four the geodesic rule reaches the goal, which is the half of the result the traps cannot establish, and on all four it produces the *same* trajectory as the projection rule to within the width of the drawn line. In the zigzag it weaves through the three staggered baffles in 63.0 s and 25.4 m, exactly as the projection rule does; in the warehouse, the open corridor and the single door the two runs agree to three decimals. Where the straight line is already right, the geodesic field agrees with it, and the machinery costs nothing in behaviour. This is the property that the traps alone could not have shown, and it is what distinguishes the rule from simply refusing concavities.

The memory term. Equation (16) carries a second ingredient, a tabu penalty on cells already visited without progress, and it was expected to be the other half of the repair. It is not, and there are two measured reasons. The first is that its stall detector never fires at the deployed sampling time: the trail is trimmed of every sample older than the window, so at $\Delta t = 0.35$ s it spans 9.80 s against a 10 s threshold and the comparison can only succeed when the window happens to be a multiple of the step. With that corrected the penalty arms once on *long wall, gap south*, and the run is unchanged for every weight from 1 to 10: same trajectory, same length, same clearance. The second reason is why. The candidates lie on the boundary of the window, which is by construction the frontier of the region the robot has *not* visited, so the penalty evaluates to zero on precisely the cells the rule is choosing between, and what actually blocks the robot is that $A^\star$ cannot reach the cell it chose, which no penalty on the target can repair. The mechanism is also not wired into the deployed node at all; it exists in the planner and in the offline harness. It is retained, disabled, and the honest conclusion is that the metric and the reachability check were the repair and the memory was a hypothesis the data does not support.

Cost of the repair. The geodesic field is the one expensive part of the planning layer, because the wavefront covers the whole known map rather than the window alone. Over the ten worlds its median cost per replanning cycle ranges from 80 to 121 ms, with a worst case of 440 ms, against a replanning period of 1 s. It is charged entirely to the 1 Hz discrete layer and leaves the 8 Hz MPC untouched, which is the reason the repair could be afforded at all: the same decision made inside the NLP would have had to fit in a 125 ms cycle.

5.5. Weight tuning in closed loop: the per-axis sweep

The procedure of Section 4.5 was run in full [14], but on the Go2 and not on the humanoid this report deploys. The reason is availability rather than principle: the quadruped is no longer accessible, and the campaign is priced in rollouts ($11m + 1$ closed-loop missions), which there has been neither the time nor the hardware

window to repeat on the G1 alongside the horizon and kinematics studies of Section 5.3. The sweep is reported here because what it tests is the *prediction* of Section 4.5, which is platform-independent; the optima it returns are not claimed to be current.

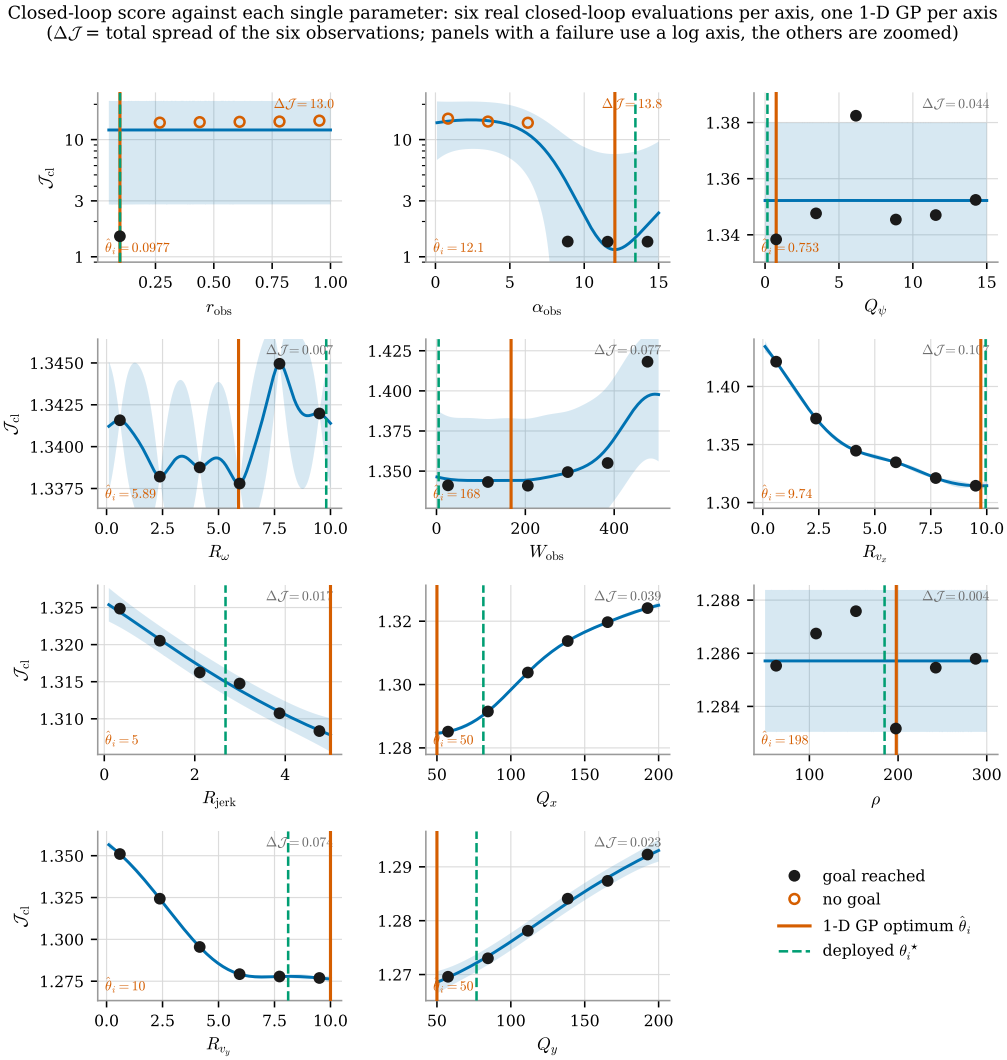


Figure 12: One-factor-at-a-time sweep: closed-loop score against each weight in turn, six rollouts and one 1-D GP per axis, with the band of (39). Filled markers reach the goal, hollow ones do not; solid line is the axis optimum (40), dashed the deployed value. $\Delta\mathcal{J}$, the spread of the six observations, is the only quantity comparable across panels; those containing a failure use a log axis and the rest are zoomed, so the apparent amplitude is not.

The result is concentrated in two axes out of eleven. The safety radius r_{obs} and the barrier sharpness α_{obs} move the score by $\Delta\mathcal{J} = 13.0$ and 13.8, against ≤ 0.10 for the other nine, better than two orders of magnitude of separation, on a criterion that needed no threshold to be visible. In both, the spread is made of missions that never reach the goal, which is the prediction of Section 4.5 confirmed: the gain is bought on the failure branch of (37) and not wrung out of a configuration that already works.

The three guards each earn their place in the same picture. R_ω and ρ_T move the score by 0.007 and 0.004, which is run-to-run scatter, and the GP returns a confident $\hat{\theta}_i$ on both regardless: without the significance floor the procedure would report eleven optima and ship nine of them as noise. Several of the surviving optima sit on an endpoint of their interval, so what the sweep located there is the bracket and not a minimum. And the two dominant axes are precisely the pair that separability treats as independent, which is why the assembled vector is re-evaluated rather than trusted.

That last point is also the substantive reason the numbers are not carried over, and it is worth separating from the logistical one. By (24) both r_{obs} and α_{obs} are tied to a footprint: r_{obs} is a clearance radius in metres and α_{obs} the sharpness of the zone it centres. The two machines do not share a footprint, so the two axes that carry the entire result are the two with the weakest claim to transfer, weaker, in fact, than the assumed time constants

of Table 1, which at least describe the same kind of quantity on both platforms. Re-running the campaign on the G1 before its kinematics and the two horizons are settled would in any case tune a configuration that is about to change. The procedure is stated in Section 4.5, and bounded in advance by the prediction it makes, so that the measurement, when it is made, is read against that prediction rather than on its own.

6. Simulation and hardware deployment

Simulation setting. Nothing was taken to the hardware without first running, unchanged, against a physics simulator: the stack is a set of ROS 2 nodes, and the simulator replaces only the two ends of the chain, the sensors that feed it and the locomotion layer that consumes its twist. Three simulators were used, and the reason there are three is that they answer different questions.

Gazebo is where the stack was first closed, on the Go2, and it is still the cheapest way to exercise the full node graph: it runs faster than real time, its worlds are built out of primitives, and its sensor plugins publish on the same topics as the drivers. Figure 13 shows one such world, a walled arena with pallets and boxes, carrying the G1 under the deployed configuration. Alongside it is the telemetry front-end used throughout the campaigns, Foxglove connected to the ROS bridge: the traces are the three components of the commanded twist $u = (v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, \omega^{\mathrm{cmd}})$ that the MPC publishes at 8 Hz, and the 3-D panel shows the accumulated LiDAR returns with the local occupancy grid of Section 2.1 rebuilt on top of them. What Gazebo cannot supply is appearance: its rendering is too synthetic to exercise anything that looks at an image.

MuJoCo and *Isaac Sim* were therefore added for the G1, and both bring a contact solver rather than a kinematic surrogate, so the reinforcement-learning gait of Section 2.3.2 is driven against the dynamics it was trained for and the twist the MPC publishes is tracked by a walking machine rather than by an integrator. MuJoCo is the lighter of the two and is what the recorded missions come from: the replay run of Section 5.1 was logged with the G1 walking a mapless MuJoCo world (Figure 14), which is why the operating points certified in Section 4.3 are contact-driven ones even though the prediction model inside the NLP is not. Isaac Sim is the photorealistic end of the range, and it is the only one whose rendering is good enough to close a perception loop on: Figure 15 shows the G1 standing in a textured warehouse among racking, pallets, cones and a forklift, a scene whose clutter is geometric and visual at once.

That photorealism is what makes the odometry front-end testable in simulation at all. The pose the stack consumes is not published by the platform: it is estimated on board by *visual–inertial odometry*, a stereo camera fused with the IMU, and it enters the graph through the odometry bridge of Section 2.1, which is where the estimate is turned into the single pose topic and differentiated into the body-frame velocity estimate that seeds $\hat{x}$ at every cycle. The right-hand panel of Figure 15 shows both halves of that arrangement running together: the visual odometry track, and the displays the navigation stack actually plans on, which are all LiDAR-derived, the marked returns with the ground plane rejected, the local voxel map, the stop and slow-down zones of the collision monitor, and the MPC prediction published each cycle. The division of labour is deliberate and worth stating plainly, because it is the one the whole report assumes: the camera localises, and the LiDAR is the only exteroceptive input that reaches the optimization programs, exactly as declared in Section 2.2. A perception failure and a planning failure therefore stay separable, which they would not be if the same sensor served both.

One caveat closes the paragraph, so that the simulators are not read as the plant of Section 5. The physics simulators are what the *deployed* stack runs against; the counterfactual campaigns of that section deliberately do not use them, because with the prediction model equal to the simulation model those experiments measure the optimizer rather than the gait. The simulators answer the complementary question, whether the stack survives contact, latency and a real perception front-end, and it is in that role that they are reported here.

Hardware deployment. The stack was deployed on both machines, the Go2 and the G1, and on both it runs on an NVIDIA Jetson Orin Nano carried by the robot, as a companion computer rather than a replacement for the vendor controller: the internal PC keeps the sensors and the locomotion layer, and the Jetson runs the ROS 2 nodes of Section 2.1. The two exchange over DDS, the transport ROS 2 already speaks, so the board subscribes to the LiDAR returns and to the state published by the internal PC and publishes back the body-frame twist u that Section 2.2 declares as the common command interface of the two platforms. Nothing in the optimization program crosses that boundary differently: the deployed profile `planner_params_g1.yaml`, the horizon $N = 15$ at $\Delta t = 0.35$ s and the solver options of Section 4.4 are the ones analysed throughout this report.

On model mismatch, and why it was not analysed further. The prediction model of Section 2.5 is a planar holonomic agent standing in for a walking machine, so a systematic divergence between the predicted and the executed trajectory is the failure this stack should be most exposed to. On both platforms it did

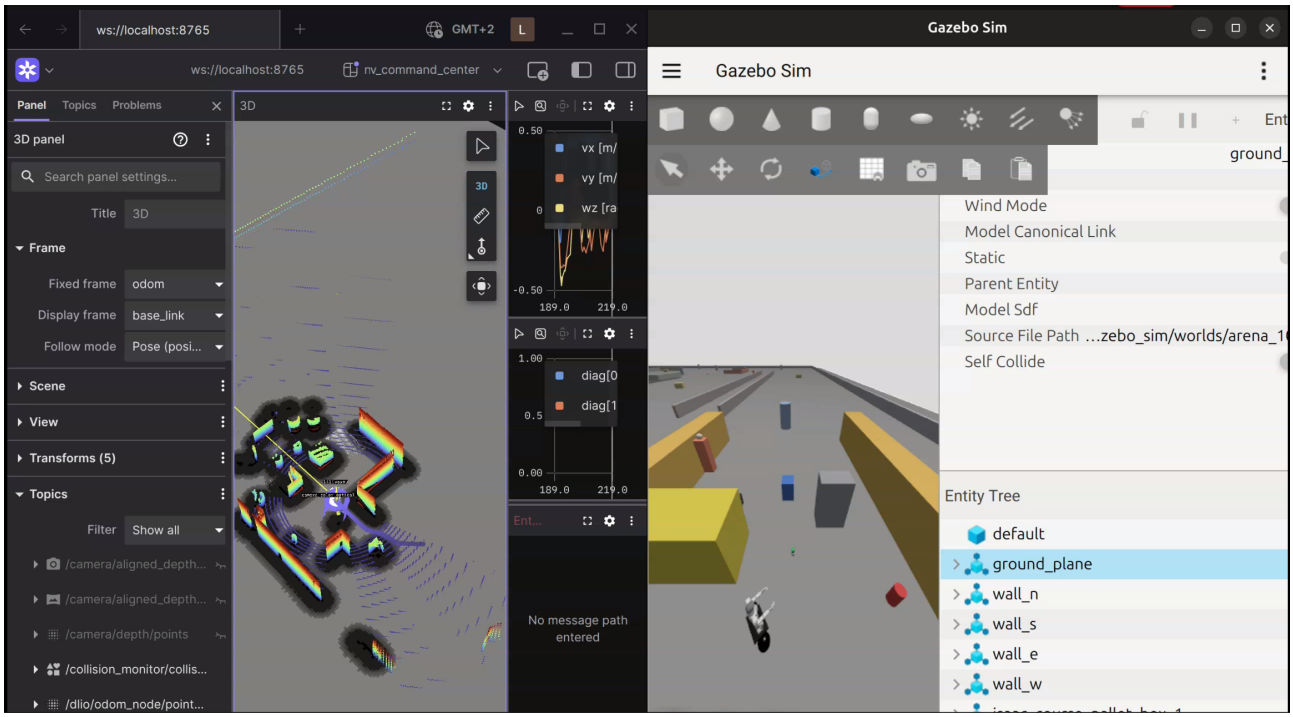


Figure 13: Gazebo (right), where the stack was first closed on the Go2 and which now carries the G1 in scripted arena worlds, with the Foxglove front-end used for telemetry (left). The traces are the three components of the twist commanded by the MPC; the 3-D panel shows the accumulated LiDAR returns and the local occupancy grid rebuilt from the newest scan. Gazebo is the fastest of the three simulators and the only one routinely run faster than real time, which is what makes it the one used for regression runs of the whole node graph.

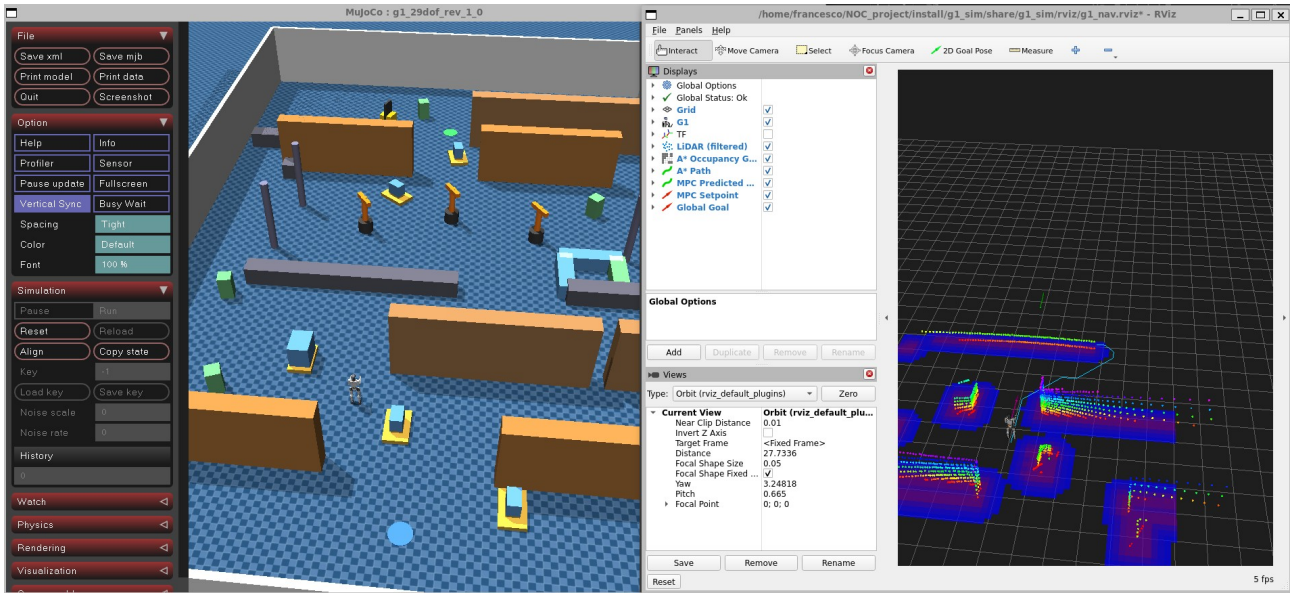


Figure 14: The G1 in MuJoCo (left), the lighter of the two dynamic simulators and the one the replay bag of Section 5.1 was recorded in. Rendering is deliberately minimal: what MuJoCo is used for is the contact solver, so that the twist published by the MPC is tracked by the gait policy under the dynamics it was trained on. The RViz panel (right) shows the same run from the stack's point of view, with the filtered LiDAR returns, the local occupancy grid, the $A^\star$ path and the MPC predicted trajectory and setpoint drawn on top of each other.

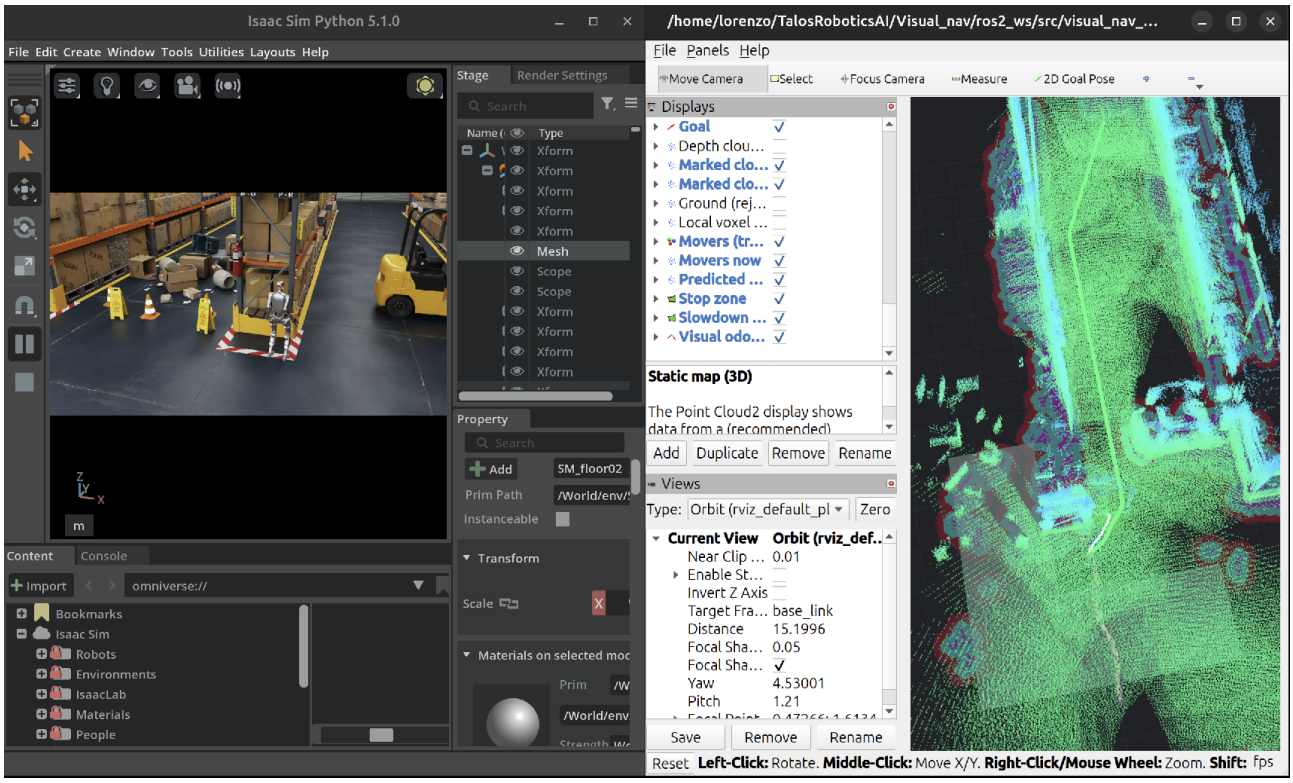


Figure 15: The G1 in Isaac Sim (left) and the navigation stack running on it (right). The simulated scene supplies both the contact dynamics for the gait policy of Section 2.3.2 and the rendered stereo pair from which the visual–inertial odometry is computed. In the RViz view every display the planner consumes is LiDAR-derived: the marked returns with the ground rejected, the local voxel map, the stop and slow-down zones, and the MPC prediction of Section 3.4; the visual odometry appears as a separate track, feeding the pose bridge of Section 2.1 rather than the map.

not appear: across the deployment runs the robot tracked the reference without the persistent lateral drift or the accumulating along-track error an unmodelled bias would produce, and no run had to be stopped for the executed path leaving the planned corridor. Part of the reason is that the pose the tracker consumes is not dead-reckoned. The deployed estimator is a LiDAR-inertial odometry, an extended Kalman filter fusing the joint encoders and the inertial unit with the LiDAR scans, so the estimate is continuously corrected against observed geometry rather than integrated in open loop, and the drift a purely proprioceptive estimate would accumulate over a multi-minute mission is bounded by the structure the sensor keeps seeing. The visual–inertial front-end of Figure 15 is not what the deployed stack runs on: it is shown to make the point that the same stack closes without a LiDAR at all, the camera taking over localisation while the rest of the graph is untouched. A quantitative mismatch campaign, inflating the plant parameters against the model and measuring what the closed loop loses, would put a number on the margin this leaves, and is the natural continuation; it is not what the deployment evidence asked for.

7. Conclusions

This report analysed a deployed map-free navigation stack as what it is numerically: an exact $A^\star$ search on a Gaussian occupancy grid, a 141-variable nonlinear program with a 1.73% dense constraint Jacobian, and a set of design decisions each of which has a defensible alternative in the theory.

The decisions that survived measurement are worth stating as such. Algorithmic differentiation costs 1.46 objective evaluations against 282 for central differences and is exact, which is what makes the exact Hessian affordable and buys 61% of the iterations. The interior-point method is faster than the active-set alternative in both regimes into which this problem can be put, and its margin widens with the number of inequalities exactly as expected. And the first- and second-order optimality conditions hold at every cycle examined, with unique multipliers.

Three results contradicted the design, and they are the ones that taught the most. The mid-point rule is 114× more accurate than the scheme it replaced and changes the closed loop by essentially nothing, because the prediction is limited by the model and not by its integration. The deployed horizon is dominated: a configuration a quarter of the size matches it on task performance at a third of the tail solve time. And the input set, which on the narrow envelope of the earlier campaigns was the constraint that decided what the controller could do, has stopped being one: widened to $|v_y| \leq 0.20$ m/s and $v_x \in [-0.20, 0.40]$ m/s, it leaves the critical cone at 43 of 141 dimensions at the last sampled cycle, and the controller is cost-driven to the end.

Three properties of the stack matter beyond the numbers above, because they say what it is good for rather than how well it performs. It is multi-robot by construction, not by retrofit: the same planner, NLP, and LiDAR-inertial estimator ran unmodified on the Go2 quadruped and the G1 humanoid, differing only in the deployed profile of Section 4.4, because both platforms expose nothing to the controller but the body-frame twist u of Section 2.2. It closes the same graph in simulation and on hardware: the Isaac Sim pipeline of Figure 15 drives the identical ROS 2 nodes of Section 2.1 that the Jetson runs onboard, so nothing about the controller changes between a run that is replayed and one that is not. And none of it depends on an a priori map or a SLAM back end: the planner searches a rolling occupancy grid built from the live LiDAR or stereo return, and the pose is corrected by a local built occupancy grid. That last property is what makes the stack worth deploying outside a benchmark course: it is a candidate wherever building and keeping a map is the actual obstacle, planetary and other exploration settings where no prior map exists to build against, and general outdoor navigation where the terrain changes faster than a map of it stays valid.

The future work involves: Identify the actuator time constant under physics, which unblocks both the sampling-time analysis and an honest re-measurement of the terminal constraint. Apply $u_0^\star$ directly instead of through a look-ahead setpoint, so that improvements provable in the plan become observable in the trajectory. And revisit the input envelope before the cost weights, because that is where the evidence points.

References

[1] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *18th European Control Conference (ECC)*, pages 3420–3431, 2019.

[2] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi – a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

[3] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Wiley, 2001.

[4] Hong Chen and Frank Allgöwer. A quasi-infinite horizon nonlinear model predictive control scheme with guaranteed stability. *Automatica*, 34(10):1205–1217, 1998.

[5] Moritz Diehl, Rolf Findeisen, Frank Allgöwer, Hans Georg Bock, and Johannes P. Schlöder. Nominal stability of real-time iteration scheme for nonlinear model predictive control. In *IEE Proceedings – Control Theory and Applications*, volume 152, pages 296–308, 2005.

[6] Lorenzo Fagiano. Numerical optimization for control, 2025. Lecture notes, course 062047, M.Sc. in Automation and Control Engineering, Politecnico di Milano, v1.1.

[7] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[8] Dong Jin Hyun, Sangok Seok, Jongwoo Lee, and Sangbae Kim. High speed trot-running: implementation of a hierarchical controller using proprioceptive impedance control on the MIT Cheetah. *The International Journal of Robotics Research*, 33(11):1417–1445, 2014.

[9] Juan Miguel Jimeno. CHAMP: Hierarchical controller for highly dynamic quadrupedal locomotion. `https://github.com/chvmp/champ`, 2020.

[10] Sven Koenig and Maxim Likhachev. Fast replanning for navigation in unknown terrain. *IEEE Transactions on Robotics*, 21(3):354–363, 2005.

[11] Jongwoo Lee. Hierarchical controller for highly dynamic locomotion utilizing pattern modulation and impedance control: implementation on the MIT Cheetah robot. M.Sc. thesis, Massachusetts Institute of Technology, Department of Mechanical Engineering, Cambridge, MA, USA, 2013. `https://dspace.mit.edu/handle/1721.1/85490`.

[12] Jialong Li, Xuxin Cheng, Tianshu Huang, Shiqi Yang, Ri-Zhao Qiu, and Xiaolong Wang. AMO: Adaptive motion optimization for hyper-dexterous humanoid whole-body control. In *Proceedings of Robotics: Science and Systems (RSS)*, 2025. `https://arxiv.org/abs/2505.03738`.

[13] David Q. Mayne, James B. Rawlings, Christopher V. Rao, and Pierre O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.

[14] Lorenzo Ortolani, Gabriel Voss, Gabriele Beltrami, Francesco Dorati, and Tommaso Felice Banfi. Bayesian optimization for learning nonlinear MPC in autonomous agent navigation. Talos Robotics AI, Milan, Italy. Companion paper of the `Go2_navigation` project, 2026.

[15] Gabriele Pannocchia and James B. Rawlings. Disturbance models for offset-free model-predictive control. *AIChE Journal*, 49(2):426–437, 2003.

[16] Marc H. Raibert. *Legged Robots That Balance*. MIT Press, Cambridge, MA, USA, 1986.

[17] Carl E. Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[18] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 edition, 2017.

[19] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 91–100. PMLR, 2022. `https://proceedings.mlr.press/v164/rudin22a.html`.

[20] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017. `https://arxiv.org/abs/1707.06347`.

[21] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.

[22] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.

[23] Stephen J. Wright. Coordinate descent algorithms. *Mathematical Programming*, 151(1):3–34, 2015.